

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Ярославский государственный университет имени П. Г. Демидова»

Кафедра дискретного анализа

Сдано на кафедру

«_____» _____ 2026 г.

Заведующий кафедрой,

д. ф.-м. н., доцент

_____ А. В. Николаев

Выпускная квалификационная работа

**Разработка игрового приложения
«Коридор» с модулем стратегического поведения
соперника**

по направлению

01.04.02 Прикладная математика и информатика

Научный руководитель

д. ф.-м. н., доцент

_____ А. В. Николаев

«_____» _____ 2026 г.

Студент группы ИВТ-21МО

_____ А. О. Карпунин

«_____» _____ 2026 г.

Ярославль, 2026

Реферат

Объем 85 с., 4 гл., 7 рис., 8 табл., 5 источников, 6 прил.

Ключевые слова: игры, Коридор, поиск кратчайшего пути, игровой искусственный интеллект, Минимакс, альфа-бета отсечение, Монте-Карло, функция оценки, обучение весов.

Объект исследования — логическая игра «Коридор» и алгоритмы автоматического расчета лучшего хода в ней.

Цель работы — разработка игрового приложения, с помощью которого пользователь сможет сыграть против одного из четырёх программных соперников, а также сравнить эти алгоритмы между собой.

Результат работы — приложение, реализующее правила игры «Коридор», с возможностью игры против компьютера или наблюдения за игрой алгоритмов друг против друга.

Содержание

Введение	5
1. Игра «Коридор»	7
1.1. Необходимые определения	7
1.2. Описание игры	7
1.3. Модуль стратегического поведения соперника	8
2. Алгоритмы	9
2.1. Общие функции и структура алгоритмов	9
2.2. Случайный алгоритм	13
2.3. Минимакс	14
2.4. Альфа-бета отсечения	16
2.5. Монте-Карло	17
2.5.1. Выбор	18
2.5.2. Расширение	19
2.5.3. Симуляция	19
2.5.4. Обратное распространение	19
2.5.5. MCTS	20
2.6. Сводные параметры сложности	20
3. Игровое приложение	22
3.1. Техническая реализация	22
3.2. Взаимодействие с приложением	24
4. Сравнение алгоритмов	28
4.1. Методика сравнения	28
4.2. Собираемые метрики	28
4.3. План эксперимента	28
4.4. Форма представления результатов	29
4.5. Анализ результатов	30
Заключение	31
Список литературы	32
Приложение А. Реализация интерфейса алгоритма	33
Приложение Б. Реализация случайного алгоритма	48

Приложение В. Реализация алгоритма MiniMax	51
Приложение Г. Реализация алгоритма AlphaBeta	58
Приложение Д. Реализация алгоритма Monte Carlo	70
Приложение Е. Реализация обучения весов функции оценки	80

Введение

Коридор — настольная логическая игра для двух или четырёх игроков, выпущенная в 1997 году компанией Gigamic. Она была создана Мирко Маркези на основе более ранней игры «Блокада». Игра получила заметное распространение среди настольных логических игр; в частности, она входила в подборку Games 100 журнала *Games* [1].

Общий вид партии показан на рисунке 1. На каждом ходу игрок выбирает одно из следующих действий: продвинуть свою фишку к противоположной стороне поля или поставить стену, которая усложнит маршрут соперника. Количество таких стен ограничено, а правила игры запрещают полностью блокировать путь любого игрока к его целевой стороне поля.

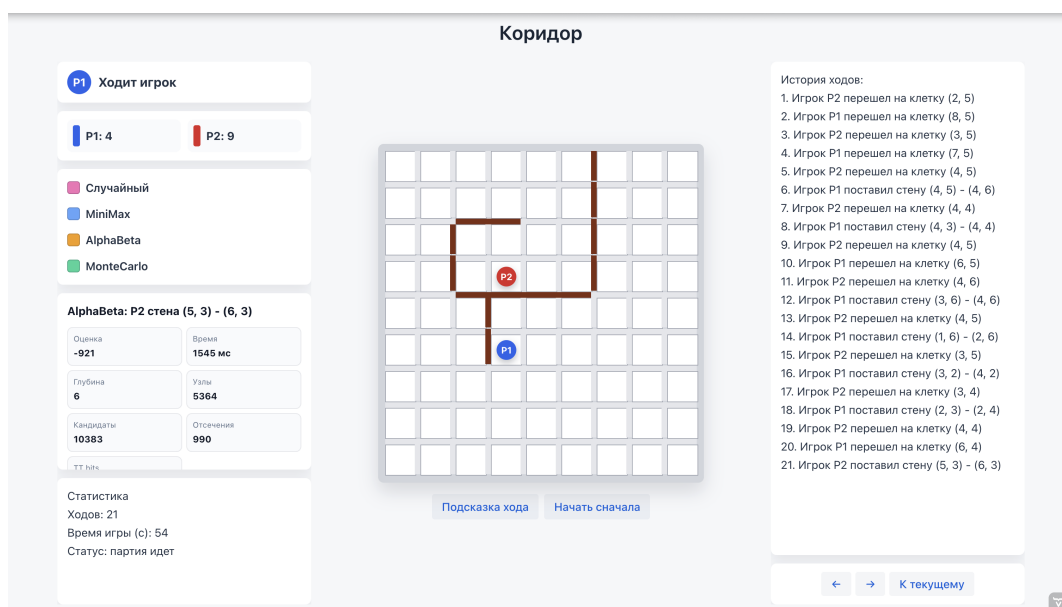


Рис. 1 — Пример партии в приложении «Коридор»

Целью выпускной квалификационной работы является разработка игрового приложения «Коридор», включающего реализацию правил игры, визуализацию игрового поля и модуль стратегического поведения, объединяющий четыре алгоритма программного соперника.

Для реализации цели работы были поставлены следующие задачи:

1. изучить правила игры «Коридор»,
2. реализовать алгоритм проверки корректности перемещения фишки,
3. реализовать алгоритм проверки корректности установки перегородки с проверкой достижимости целевых сторон поля,
4. реализовать алгоритм поиска кратчайшего пути до целевой стороны поля на основе обхода графа в ширину,

5. разработать функцию оценки игровой позиции с настраиваемыми весами и механизмом их коррекции по результатам партий,
6. реализовать четыре алгоритма программного соперника: случайный выбор хода, минимакс, альфа-бета отсечение и Монте-Карло,
7. провести сравнение алгоритмов в серии автоматических партий,
8. разработать интерфейс игрового приложения.

1. Игра «Коридор»

1.1. Необходимые определения

В формулировке правил игры и последующем описании алгоритмов встречается несколько понятий, которые нуждаются в пояснении.

Игровым полем называется квадратная доска размером $n \times n$ клеток, по которым перемещаются фишки игроков. В стандартном варианте игры $n = 9$, однако приложение также поддерживает поля 7×7 и 11×11 . Каждая из клеток нумеруется парой индексов (i, j) , где $0 \leq i, j \leq n - 1$.

Фишкой называется объект, который занимает одну клетку игрового поля и способен к перемещению на одну из смежных клеток.

Стеной называется элемент, который занимает пространство между клетками. Такая перегородка имеет длину две клетки и может быть расположена горизонтально, блокируя вертикальный проход между двумя парами клеток, или вертикально. Количество стен у каждого игрока ограничено: 8 для поля 7×7 , 10 для 9×9 , 12 для 11×11 .

Целевой линией игрока называется строка игрового поля, которая находится на противоположной стороне от его стартовой позиции. В начале партии игроки расположены на центрах противоположных сторон поля.

1.2. Описание игры

В начале партии фишки игроков расставляются на противоположных сторонах поля: первый игрок — в центре нижней строки, второй — в центре верхней строки. Игроки ходят по очереди, первенство определяется случайно. Цель каждого игрока — первым достичь любой клетки своей целевой линии.

За один ход игрок может выполнить только одно из двух действий [2]:

1. Перемещение по игровому полю. Фишка игрока передвигается на смежную клетку, если такой маневр не заблокирован стеной. Если соседняя клетка занята фишкой соперника и за ней нет ни перегородки, ни границы поля, фишка игрока может перепрыгнуть через соперника. Если прямой прыжок невозможен, допускается прыжок на клетку слева или справа от той, куда изначально предполагался прыжок.
2. Установка стены. Она допускается, если: у игрока есть хотя бы одна перегородка в его запасе, она не пересекается с уже установленными стенами на игровом поле и после её установки у каждого из игроков остается хотя бы один маршрут к своей целевой линии.

Игра завершается, когда фишка одного из игроков достигает любой клетки целевой линии — этот игрок побеждает.

1.3. Модуль стратегического поведения соперника

Модуль стратегического поведения — это программный компонент, который отвечает за выбор хода соперника. Основываясь на полученных параметрах, а именно текущем состоянии игрового поля, количество оставшихся стен у игроков, модуль возвращает лучший по собственной оценке ход.

В процессе разработки к модулю стратегического поведения соперника были выдвинуты следующие требования:

1. *Корректность*: выбранный алгоритмом ход не должен противоречить правилам игры.
2. *Завершённость*: алгоритм должен завершаться за конечное время. Все реализованные в приложении алгоритмы работают с ограничением по времени: каждый из них получает лимит в миллисекундах и прекращает поиск по его истечении. Этот устанавливаемый лимит зависит от сложности игры и выбранного размера игрового поля.
3. *Единое взаимодействие с выбранным алгоритмом*: все алгоритмы реализуют единый программный интерфейс `Algorithm` с методом получения лучшего хода. Благодаря этому переключение алгоритмов или добавление новых не требует вносить какие-либо изменения в остальные компоненты игрового приложения.
4. *Настраиваемая сложность*: каждый из алгоритмов поддерживает три уровня сложности — EASY, MEDIUM и HARD. Выбранный уровень сложности влияет не только на ограничение по времени, но и на глубину поиска или число итераций.

В рамках данной работы в модуле поведения соперника реализовано четыре алгоритма, каждый из которых может быть выбран в качестве оппонента для игры против пользователя. Случайный алгоритм равновероятно выбирает один из множества допустимых ходов. Минимакс перебирает дерево игровых состояний на ограниченную глубину без каких-либо отсечений. Альфа-бета расширяет минимакс, добавляя в изначальный алгоритм отсечения безперспективных ветвей дерева. Метод Монте-Карло оценивает позиции через многократные случайные симуляции игры из каждой из них.

2. Алгоритмы

2.1. Общие функции и структура алгоритмов

Каждый из разработанных алгоритмов реализуют единый программный интерфейс `Algorithm`, содержащий как абстрактные методы, тело которых определяется выбранным алгоритмом, так и конкретные реализации общих вспомогательных функций. Благодаря этому исключается дублирование кода, упрощается добавление нового алгоритма в приложение, а также гарантируется единообразное поведение алгоритмов в одних и тех же ситуациях.

Генерация допустимых ходов

Метод `getMoves(board, playerId, wallsLeft)`, который применяется всеми алгоритмами в модуле стратегического поведения соперника, формирует полное множество допустимых ходов из текущего состояния из двух частей.

Первая часть — допустимые перемещения фишки, получаемые с помощью метода `board.getAvailableMoves(pos)`, который поочередно проверяет четыре направления перемещения с текущей клетки, а также обрабатывает прыжки через соперника, если такие возможны.

Вторая часть — допустимые установки стен. Если лимит стен игрока исчерпан, эта часть не дает никаких дополнительных вариантов ходов. В противном случае алгоритм формирует множество «кандидатных» клеток, ограничивая полный перебор стратегически значимой областью поля. Кандидатами являются клетки, которые расположены около кратчайшего пути соперника к его целевой строке поля. Для каждой такой клетки проверяются два возможных направления стены. Перегородка включается в список кандидатов, если она не имеет коллизий с уже расставленными перегородками на поле, и не отрезает ни одного из игроков от его целевой линии. Помимо прочего проверяется выполнение хотя бы одного условия:

1. стена увеличивает кратчайший путь соперника минимум на 2 хода
2. стена увеличивает кратчайший путь соперника не меньше, чем собственный путь алгоритма

Итоговый список возможных стен сортируется по убыванию функции `wallImpact`.

Поиск кратчайшего пути

Метод `calculateShortestPath(board, start, targetRow)` основан на обходе графа в ширину от текущей позиции игрового поля до ближайшей

клетки целевой строки.

Метод `getShortestPathCells` дополняет предыдущий восстановлением кратчайшего пути, поскольку при поиске такового хранит в истории посещенные клетки. Как уже описывалось ранее, он используется для поиска «кандидатных» клеток для установки стен.

Эндшпильные правила

Метод `findEndgameMove` вызывается в любом из алгоритмов в качестве первого шага перед запуском основного поиска лучшего хода. Если с текущей клетки поля фишку игрока можно переместить прямо его на целевую линию — этот ход немедленно возвращается без перебора других вариантов. Если такого «победного» хода нет, но соперник находится не более чем в двух ходах от победы, а лимит стен у алгоритма не исчерпан — ищется стена с максимальным значением `wallImpact`, чтобы замедлить игрока.

Предпочтение направления движения

С помощью метода `movementPreference` к оценке хода добавляется бонус или штраф в зависимости от направления перемещения и истории предыдущих ходов:

- ход в сторону целевой линии: +45 очков;
- ход назад: −120 очков;
- уменьшение длины пути до целевой линии на Δ клеток: $+35 \cdot \Delta$ очков;
- перемещение на позицию, которая встречалась в последних двух ходах: −500 очков;
- повтор более давнего перемещения: −180 очков.

Вся история перемещений хранится для каждого игрока отдельно и получается алгоритмом путем вызова `getRecentPositions`.

Функция оценки позиции

Функция оценки `evaluate()` является главным компонентом всех реализованных в игровом приложении алгоритмов. Благодаря ей каждому состоянию игры сопоставляется числовое значение, которое описывает его стратегическую выгоду для программного соперника. Если состояние игры подразумевает победу алгоритма или пользователя функция возвращает $+\text{WIN_SCORE} + d$ или $-\text{WIN_SCORE} - d$ соответственно, где $\text{WIN_SCORE} = 100\,000$, а d — текущая глубина поиска. С помощью этого параметра получается обеспечить предпочтение более близкой победы.

Для остальных состояний игры оценка вычисляется как взвешенная сумма следующих параметров:

$$F(s) = \sum_{k=1}^7 \lambda_k \cdot f_k(s),$$

где λ_k — веса, а $f_k(s)$ — значения параметров. Параметры и их описание перечислены в таблице 2.1.

Таблица 2.1

Параметры оценочной функции

k	Признак $f_k(s)$	Описание
1	$d_p - d_a$	Разность между длинами кратчайших путей алгоритма и пользователя
2	mob_a	Количество допустимых ходов фишкой алгоритма — его мобильность
3	$-\text{mob}_p$	Отрицательное количество допустимых ходов пользователя
4	$w_a - w_p$	Преимущество алгоритма по числу оставшихся стен
5	$\text{prog}_a - \text{prog}_p$	Разность продвижений к целевым линиям алгоритма и пользователя
6	$e(d_a)$	Эндшпильный бонус за близость алгоритма к победе над игроком
7	$-e(d_p)$	Эндшпильный штраф за близость пользователя к победе над алгоритмом

Продвижение prog_a равно количеству строк игрового поля, которые прошла фишка алгоритма от стартовой позиции до текущей. Эндшпильная функция $e(d)$ может принимать такие значения:

$$e(d) = \begin{cases} 10, & d \leq 1, \\ 4, & d = 2, \\ 1, & d = 3, \\ 0, & d > 3. \end{cases}$$

Веса λ_k , соответствующие описанным параметрам, по умолчанию заданы следующим образом (класс `EvaluationWeights`):

$$\lambda_1 = 65, \quad \lambda_2 = 4, \quad \lambda_3 = 3, \quad \lambda_4 = 18, \quad \lambda_5 = 6, \quad \lambda_6 = 165, \quad \lambda_7 = 175.$$

Коррекция весов

Игровое приложение поддерживает механизм автоматической коррекции параметров функции оценки, анализируя результаты сыгранных партий (класс `EvaluationWeightsStore`). После завершения каждой из партий, в которых участвовал программный соперник, вычисляется оценка полезности ходов, которые предпринимал алгоритм.

В течение игры для каждого хода алгоритма формируется объект `EvaluationLearning` который хранит вектор параметров до хода и после него. После завершения партии для каждого образца вычисляется дельта параметров оценки $\Delta f_k = f_k^{\text{after}} - f_k^{\text{before}}$ и суммируется в градиент с учётом знака:

$$g_k = \sum_{\text{ходы алгоритма}} \text{reward} \cdot \Delta f_k, \quad \text{reward} = \begin{cases} +1, & \text{алгоритм победил,} \\ -1, & \text{алгоритм проиграл.} \end{cases}$$

Если $g_k > 0$, увеличение веса k -го параметра чаще встречалось в ходах победителя, поэтому вес увеличивается. Если $g_k < 0$, этот признак чаще участвовал в принятии решений, приводящих к проигрышу, поэтому соответствующий ему вес уменьшается. Если $g_k = 0$, вес не изменяется. Для начала необходимо вычислить предварительное значение обновленного веса:

$$\lambda'_k = \lambda_k + \text{sign}(g_k) \cdot \Delta\lambda.$$

Затем оно проверяется согласно допустимому диапазону:

$$\lambda_k^{\text{new}} = \begin{cases} \lambda_k^{\min}, & \lambda'_k < \lambda_k^{\min}, \\ \lambda_k^{\max}, & \lambda'_k > \lambda_k^{\max}, \\ \lambda'_k, & \lambda_k^{\min} \leq \lambda'_k \leq \lambda_k^{\max}. \end{cases}$$

Здесь $\Delta\lambda = 1$ — это шаг обучения, а $\lambda_k^{\min}$ и $\lambda_k^{\max}$ — минимально и максимально допустимые значения веса. Такие ограничения были наложены на веса параметров, чтобы после серии обучающих партий с алгоритмом отдельный параметр не стал чрезмерно большим, отрицательным или слишком малым и не начал полностью подавлять остальные признаки функции оценки. Актуальные веса параметров регулярно сохраняются в файл `data/evaluation-weights.properties` и используются для функции оценки в следующем запуске приложения, что обеспечивает накопление опыта между сессиями. На момент написания работы механизм коррекции предпринял 15 обновлений весов на 1388 обучающих образцах, а текущие сохранённые веса составляют:

$$\lambda_1 = 120, \quad \lambda_2 = 7, \quad \lambda_3 = 12, \quad \lambda_4 = 8, \quad \lambda_5 = 25, \quad \lambda_6 = 186, \quad \lambda_7 = 170.$$

Сравнение изначально выбранных и обновленных весов показывает, что механизм увеличил приоритет разности путей ($\lambda_1 : 65 \rightarrow 120$), штрафа за количество возможных ходов соперника ($\lambda_3 : 3 \rightarrow 12$), продвижения к целевой строке игрового ($\lambda_5 : 6 \rightarrow 25$) и собственного эндшпильного преимущества ($\lambda_6 : 165 \rightarrow 186$), одновременно с этим уменьшив влияние преимущества по оставшимся стенам ($\lambda_4 : 18 \rightarrow 8$) на общую оценку состояния игры.

Функция wallImpact

Функция `wallImpact(board, move, playerId, size)` оценивает стратегическую ценность конкретной установки стены. Применяв ход на копии доски, она вычисляет изменения длин путей участников игры до их целевых строк:

$$\text{wallImpact} = (d_p^{\text{after}} - d_p^{\text{before}}) \cdot 100 - \max(0, d_a^{\text{after}} - d_a^{\text{before}}) \cdot 45,$$

то есть каждый шаг, увеличивающий путь соперника, оценивается в 100 единиц, а каждый шаг, удлиняющий собственный путь, штрафует 45 единицами. Стена считается стратегически ценной, если `wallImpact > 0`.

2.2. Случайный алгоритм

(RandomAlgorithm) является простейшим из возможных программных соперников. При каждом вызове метода `getMove` алгоритм первым делом применяет эндшпильное правило: если можно немедленно выиграть самому или срочно заблокировать путь для соперника, он делает это независимо от уровня сложности. Если такое правило не сработало, поведение определяется уровнем сложности.

На уровне EASY алгоритм с равной вероятностью выбирает ход из полного множества допустимых ходов.

На уровне сложности MEDIUM ходы сортируются по убыванию значения оценки хода, и случайно выбирается один из первой списка ходов, ограниченного первой его половиной.

На уровне HARD случайно выбирается уже один из трёх лучших ходов с помощью применения той же оценки.

Алгоритм 1: Случайный алгоритм

```
1  эндшпильный ход  $\leftarrow$  findEndgameMove( $s$ );
2  if эндшпильный ход найден then
3    |   return эндшпильный ход
4  end
5   $M \leftarrow$  getMoves( $s$ );
6  if уровень сложности = EASY then
7    |    $m^* \leftarrow$  случайный элемент  $M$ ;
8  else if уровень сложности = MEDIUM then
9    |   сортировать  $M$  по убыванию evaluateAfterMove;
10   |    $m^* \leftarrow$  случайный элемент из первой половины  $M$ ;
11 else
12   |   сортировать  $M$  по убыванию evaluateAfterMove;
13   |    $m^* \leftarrow$  случайный элемент из первых трёх элементов  $M$ ;
14 end
15 return  $m^*$ 
```

2.3. Минимакс

В MinimaxAlgorithm во время хода алгоритма выбирается ход с максимальной оценкой (максимизирующий узел); на ходах пользователя предполагается выбор хода с минимальной оценкой (минимизирующий узел).

Алгоритм итеративно увеличивает глубину анализа хода: последовательно запускает поиск с глубиной $d = 1, 2, \dots, d_{\max}$ до истечения лимита времени. На каждом этапе цикла в специальное поле сохраняется лучший выбранный ход, значение которого будет возвращено в качестве ответа. Благодаря этому, даже если лимит времени оказался исчерпан ранее, чем поиск на очередной глубине подошел к концу, всегда будет допустимый ответ, который можно будет использовать как ход алгоритма.

Максимальная глубина $d_{\max}$ и лимит времени T зависят от выбранных пользователем уровня сложности алгоритма и размера игрового поля:

Таблица 2.2

Параметры минимакса

Уровень	$d_{\max}$ (поле 9×9)	T , мс (поле 9×9)
EASY	3	700
MEDIUM	4	1800
HARD	5	3600

Чтобы алгоритм успевал обрабатывать большие значения глубины поиска за отведенное время, было введено ограничение на количество ходов для перебора на каждой итерации. После получения всего множества возможных вариантов хода из текущей позиции, эти ходы сортируются с помощью функции оценки, а

зачем уже отсортированный список обрезается первыми 16 вхождениями в него. Уже обработанные алгоритмом позиции и их оценки хранятся в таблице *cache*: в составе ключе соединены строковое представление доски, текущая глубина поиска, флаг максимизации или минимизации и количество оставшихся стен у обоих игроков. Общая схема работы алгоритма приведена в алгоритме 2.

Алгоритм 2: Минимакс с итеративным углублением

```

1  Функция minimax( $s, d, isMax$ ):
2      if в cache есть значение для ( $s, d, isMax$ ) then
3          return значение из cache;
4      end
5      if terminal( $s$ ) then
6          return терминальную оценку позиции;
7      end
8      if  $d = 0$  then
9          return evaluate( $s$ );
10     end
11      $p \leftarrow$  игрок, которому принадлежит ход в узле  $s$ ;
12      $M \leftarrow$  лучшие 16 ходов из getMoves( $s, p$ );
13     if  $isMax$  then
14          $best \leftarrow -\infty$ ;
15         foreach  $m \in M$  do
16              $best \leftarrow \max(best, \text{minimax}(\text{apply}(s, m), d - 1, \text{false}))$ ;
17         end
18     else
19          $best \leftarrow +\infty$ ;
20         foreach  $m \in M$  do
21              $best \leftarrow \min(best, \text{minimax}(\text{apply}(s, m), d - 1, \text{true}))$ ;
22         end
23     end
24     сохранить  $best$  в cache для ( $s, d, isMax$ );
25     return  $best$ ;

26  $m^* \leftarrow$  первый допустимый ход;
27 for  $d \leftarrow 1$  to  $d_{\max}$  do
28     if истёк лимит времени  $T$  then
29         break
30     end
31      $m^* \leftarrow \arg \max_{m \in M(s)} (\text{minimax}(\text{apply}(s, m), d - 1, \text{false}) +$ 
        movementPreference( $m, s$ ));
32 end
33 return  $m^*$ 

```

2.4. Альфа-бета отсечения

AlphaBetaAlgorithm является наиболее сложным из реализованных. Он представляет собой улучшенный Минимакс с несколькими независимыми механизмами ускорения: отсечениями ветвей дерева игры и сортировкой ходов [3].

Альфа-бета отсечение

Алгоритм использует два следующих параметра: α — это наилучшая оценка, которая гарантирована максимизирующему игроку, а β — это наилучшая оценка, гарантированная минимизирующему. Если в ходе перебора оказывается, что $\beta \leq \alpha$, оставшиеся ходы в данном узле дерева можно не рассматривать: они не изменят итогового выбора. Это позволяет уменьшить число рассматриваемых состояний и благодаря этому достичь большей глубины поиска хода.

Сортировка ходов

Эффективность отсечений ветвей критически зависит от порядка рассмотрения ходов: если лучший ход рассматривается первым, отсечения происходят как можно раньше, что оптимизирует затраченное время. Функция сортировки `scoreMove` оценивает каждый ход до его применения по нескольким критериям:

- $+10\,000 / +8\,000$, если ход является первым или вторым *killer-ходом* данного уровня глубины;
- вес из таблицы `goodMoves` — истории ходов, вызывавших отсечения в предыдущих узлах;
- значение функции оценки после применения хода.

Под *killer-ходом* понимается ход, который на проверке той же глубины поиска в дереве ранее уже привёл к отсечению $\beta \leq \alpha$. Такой ход считается перспективным не потому, что он обязательно является лучшим в текущей позиции, а потому что в похожих по глубине узлах он уже оказался достаточно сильным, чтобы сделать дальнейший перебор ветви бессмысленным. В реализации алгоритма для каждого уровня глубины хранится до двух таких ходов в историческом массиве `bestMoves`; при сортировке они получают высокий приоритет и рассматриваются раньше остальных.

Таблица `goodMoves` решает похожую задачу, но хранит не два последних хода для конкретной глубины, а накопленную статистику по строковому представлению ходов. Каждый раз, когда ход приводит к отсечению, его счётчик увеличивается. При следующих сортировках ход с наибольшим числом успешных отсечений получает небольшой дополнительный вес. Таким образом, `bestMoves`

отражает локальную историю по глубине поиска, а `goodMoves` — общую историю ходов, которые часто помогают быстрее отсекаать ветви дерева.

После сортировки список ходов обрезается до 36 кандидатов, по аналогии с алгоритмом Минимакс.

Поиск стабильной терминальной позиции

Обычный поиск с ограничением глубины имеет одну важную проблему: он может остановиться в момент, когда позиция ещё не является стабильной. Например, алгоритм дошёл до предельной глубины и видит, что путь до цели стал короче, но не успел проверить, что соперник следующим ходом тоже может выйти на финишную линию или поставить решающую стену. В такой ситуации простая статическая оценка может быть слишком оптимистичной и не может гарантировать победу алгоритма.

Для уменьшения этой ошибки в `AlphaBetaAlgorithm` используется поиск стабильной терминальной позиции. Его идея состоит в том, что на предельной глубине алгоритм не всегда сразу вызывает `evaluate()`, а сначала проверяет, не является ли позиция «острой». Таковой считается позиция, в которой хотя бы один из игроков находится не дальше двух ходов от победы.

Если позиция признана «острой», алгоритм выполняет небольшое дополнительное расширение дерева: рассматривает немедленные победные ходы и до восьми наиболее ценных установок стен, которые сильнее всего увеличивают путь соперника до цели. После этого для оставшихся «спокойных» позиций снова используется обычная функция оценки.

Параметры алгоритма в зависимости от уровня сложности для поля 9×9 :

Таблица 2.3

Параметры альфа-бета алгоритма

Уровень	$d_{\max}$	T , мс
EASY	4	1300
MEDIUM	6	3600
HARD	8	7600

Общая схема работы альфа-бета алгоритма приведена в алгоритме 3.

2.5. Монте-Карло

Алгоритм `MonteCarloAlgorithm` строит статистику по многократным симуляциям игры с текущей позиции фишки на игровом поле до конца партии [4]. В отличие от уже рассмотренных алгоритмов, он не перебирает дерево равномерно до фиксированной глубины, а постепенно концентрирует свои вычисления на наиболее перспективных ветвях. Структура алгоритма — дерево узлов, каждый из

Алгоритм 3: Альфа-бета

```
1 Функция alphabeta( $s, d, \alpha, \beta, isMax$ ):
2   if terminal( $s$ ) then
3     return терминальная оценка
4   end
5   if  $d = 0$  then
6     if isNoisyPosition( $s$ ) then
7       return quiescence( $s, \alpha, \beta, isMax, 2$ )
8     else
9       return  $F(s)$ 
10    end
11  end
12   $M \leftarrow$  лучшие 36 ходов из getMoves( $s$ );
13  best  $\leftarrow -\infty$  если isMax, иначе  $+\infty$ ;
14  foreach  $m \in M$  do
15     $v \leftarrow$  alphabeta(apply( $s, m$ ),  $d - 1, \alpha, \beta, \neg isMax$ );
16    обновить best,  $\alpha$  или  $\beta$ ;
17    if  $\beta \leq \alpha$  then
18      обновить killer-ходы и goodMoves;
19      break ; // отсечение
20    end
21  end
22  return best
```

которых соответствует некоторому состоянию игры и хранит счётчики посещений узла во время проверки n_v и побед w_v , которые были достигнуты с посещением этого узла.

Основной цикл алгоритма состоит из четырёх фаз.

2.5.1. Выбор

Начиная с корня, алгоритм спускается по дереву, на каждом шаге выбирая дочерний узел с максимальным значением UCT. Этот показатель нужен для баланса между двумя задачами: продолжать проверять ходы, которые уже часто приводили к победе, а также исследовать менее изученные ходы.

$$\text{UCT}(v) = \frac{w_v}{n_v} + C \cdot \sqrt{\frac{\ln n_{\text{parent}}}{n_v}} + h(v),$$

Первое слагаемое $\frac{w_v}{n_v}$ отражает уже накопленную статистику: чем чаще симуляции из узла v заканчивались победой, тем выше его оценка.

Второе слагаемое $C \cdot \sqrt{\frac{\ln n_{\text{parent}}}{n_v}}$ отвечает за исследование. Если дочерний узел посещался редко, знаменатель n_v мал, поэтому значение этой части растёт и алгоритм получает стимул попробовать такой ход ещё раз. При увеличении числа

посещений бонус постепенно уменьшается, и решение всё сильнее зависит от реальной статистики побед. Константа $C = 1,2$ задаёт силу исследования: при большем значении алгоритм активнее проверяет разные ветви, при меньшем быстрее концентрируется на уже найденных хороших ходах.

Третье слагаемое $h(v)$ является дополнительной эвристической поправкой, зависящей от функции оценки позиции:

$$h(v) = 0,15 \cdot \tanh(F(s_v)/500).$$

Она добавляет в формулу длины путей до цели, мобильность фишек игроков, оставшиеся лимиты стен стен и эндшпильные признаки. При этом коэффициент $0,15$ и функция $\tanh$ ограничивают влияние эвристики, чтобы она не подавляла статистику симуляций.

Если узел ни разу не посещался, ему присваивается приоритет $+\infty$. Благодаря этому каждый новый допустимый ход хотя бы один раз попадает в симуляцию, и алгоритм не отбрасывает вариант только потому, что по нему ещё нет статистики.

2.5.2. Расширение

Когда все допустимые ходы из узла уже рассматривались в дочерних узлах, выбранный узел помечается как полностью расширенный. Иначе из списка ещё не рассмотренных ходов (предварительно отсортированных по `scoreMove`) берётся первый, создаётся новый дочерний узел и начинается его рассмотрение.

2.5.3. Симуляция

Из нового узла стартует случайная игра до терминального состояния или до исчерпания лимита шагов `rolloutDepth`. На каждом из ходов текущей симуляции применяется следующий механизм:

1. если есть победный ход, немедленно завершающий партию — используется он;
2. если соперник находится не далее 2 ходов от победы и у текущего игрока еще остались стены в запасе выбирается стена с максимальным `wallImpact`;
3. с вероятностью $0,22$ выбирается случайный ход из полного списка;
4. иначе из случайной выборки размером 18 ходов выбирается один из четырёх лучших по `scoreRolloutMove`.

2.5.4. Обратное распространение

Результат сыгранной симуляции распространяется вверх: для каждого узла на пути от текущей позиции к корню дерева проставляются актуальные значения n_v и w_v .

2.5.5. MCTS

После истечения лимита времени или итераций алгоритма выбирается ход с наибольшим значением $n_c + w_c/n_c$ среди дочерних узлов корня, что отдаёт предпочтение хорошо исследованным ходам.

Параметры алгоритма для поля 9×9 :

Таблица 2.4

Параметры MCTS

Уровень	Бюджет итераций	T , мс	Глубина симуляции
EASY	520	950	$9 \cdot 4 = 36$
MEDIUM	1700	2900	$9 \cdot 7 = 63$
HARD	3400	6200	$9 \cdot 10 = 90$

Алгоритм 4: MCTS

```
1  $r \leftarrow$  корневой узел с состоянием  $s$ ;  
2 while не истёк лимит  $T$  и итераций  $< N$  do  
3    $v \leftarrow \text{select}(r)$ ; // спуск по UCT  
4   if  $\neg \text{terminal}(v.s)$  then  
5      $v \leftarrow \text{expand}(v)$ ; // новый дочерний узел  
6   end  
7    $\text{result} \leftarrow \text{simulate}(v, \text{rolloutDepth})$ ;  
8    $\text{backpropagate}(v, \text{result})$ ;  
9   if  $r.n > 300$  и  $\max_c(w_c/n_c) > 0.97$  then  
10    break  
11  end  
12 end  
13 return  $\arg \max_{c \in \text{children}(r)} (n_c + w_c/n_c)$ 
```

Условие досрочного выхода из узла дерева ($\text{bestWinRate} > 0.97$ при более чем 300 посещениях корня) позволяет экономить время в позициях, где один из ходов явно доминирует по статистике.

2.6. Сводные параметры сложности

Для удобства настройки экспериментов сравнения алгоритмов все уровни сложности сведены в таблицу 2.5. Значения приведены для стандартного поля 9×9 как базовые параметры приложения. Для полей 7×7 и 11×11 параметры масштабируются в коде: на малом поле AlphaBeta может увеличивать глубину поиска, а на большом поле MiniMax и AlphaBeta уменьшают глубину либо увеличивают временной лимит, чтобы партия оставалась интерактивной.

Таблица 2.5

Параметры уровней сложности алгоритмов для поля 9×9

Алгоритм	Уровень	Параметры
Random	EASY	Случайный выбор среди всех допустимых ходов
Random	MEDIUM	Случайный выбор из верхней половины ходов по функции оценки
Random	HARD	Случайный выбор из трёх лучших ходов по функции оценки
MiniMax	EASY	Глубина 3, лимит 700 мс, ширина перебора до 16 ходов
MiniMax	MEDIUM	Глубина 4, лимит 1800 мс, ширина перебора до 16 ходов
MiniMax	HARD	Глубина 5, лимит 3600 мс, ширина перебора до 16 ходов
AlphaBeta	EASY	Глубина 4, лимит 1300 мс, сортировка ходов, до 36 кандидатов
AlphaBeta	MEDIUM	Глубина 6, лимит 3600 мс, приоритет ходов, ранее приводивших к отсечениям
AlphaBeta	HARD	Глубина 8, лимит 7600 мс, поиск стабильности позиции в острых состояниях
MCTS	EASY	До 520 итераций, лимит 950 мс, глубина rollout $9 \cdot 4 = 36$
MCTS	MEDIUM	До 1700 итераций, лимит 2900 мс, глубина rollout $9 \cdot 7 = 63$
MCTS	HARD	До 3400 итераций, лимит 6200 мс, глубина rollout $9 \cdot 10 = 90$

3. Игровое приложение

3.1. Техническая реализация

Игровое приложение «Коридор» написано на языке программирования Java версии 17 с использованием фреймворка Spring Boot версии 3.2.5, а также библиотеки Vaadin версии 24.4.2 [5], которая отвечает за веб-интерфейс. Для сокращения шаблонного кода используется библиотека Lombok. Система сборки кода — Maven.

Архитектура приложения разделена на несколько независимых слоёв. Общая схема взаимодействия слоёв представлена на рисунке 2.

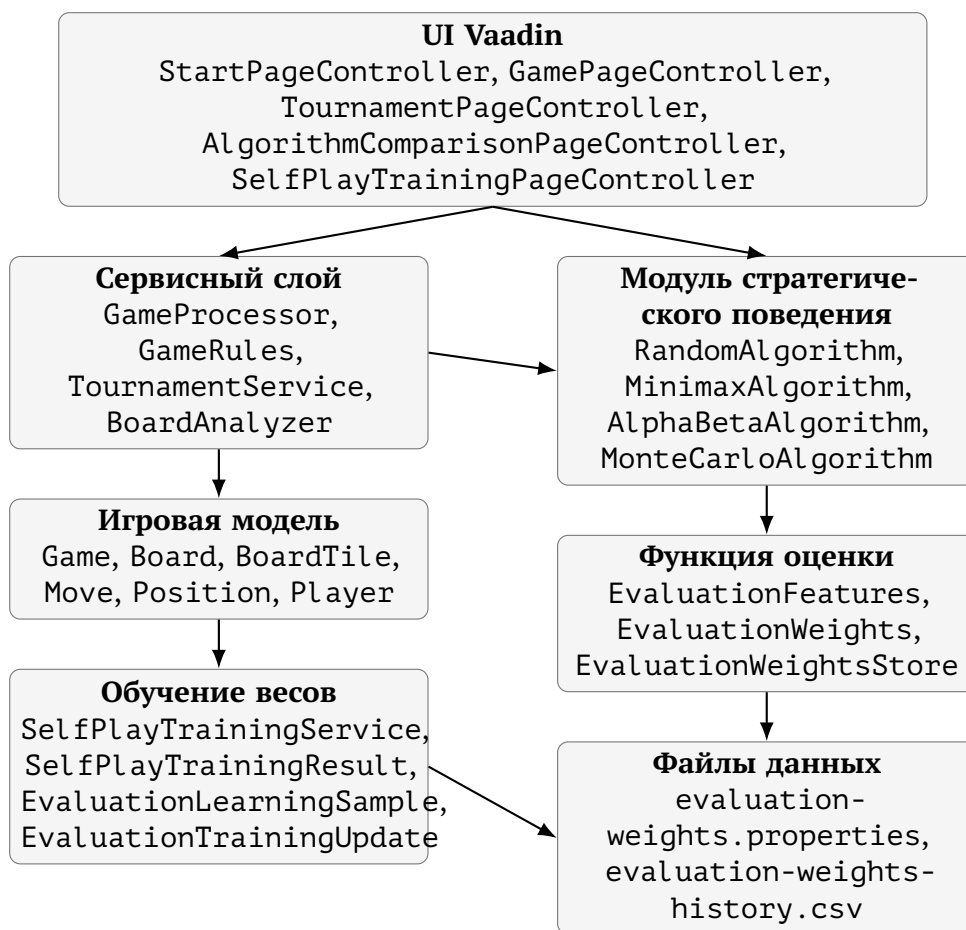


Рис. 2 — Общая архитектура приложения

Модель (`ru.ac.uniyar.model`) содержит классы, описывающие игровые объекты. Класс `Position` хранит координаты клетки (i, j) и предоставляет метод `move(rowDelta, colDelta)` для вычисления соседней позиции. Класс `BoardTile` описывает одну клетку поля четырьмя булевыми флагами доступности переходов. Класс `Board` хранит граф поля в виде `HashMap<Position, BoardTile>`, позиции фишек обоих игроков и реализует все операции с доской: размещение

перегородок (`placeWall`), получение доступных ходов (`getAvailableMoves`) и полное копирование (`copy`). Класс `Move` описывает ход: его тип (`MOVE_PLAYER` или `PLACE_WALL`), идентификатор игрока и позиции начала и конца перемещения или стены. Класс `Game` хранит конфигурацию текущей партии: размер игрового поля (`GameSize`), объекты игроков, номер текущего хода, время начала игры и флаг ее завершения.

Игроки (`ru.ac.uniyar.model.players`) реализуют иерархию через абстрактный класс `Player` с методом `getMove`. Класс `HumanPlayer` возвращает `null`, поскольку ход игрока получается через его взаимодействие с интерфейсом игрового приложения; класс `ComputerPlayer` делегирует вызов конкретному объекту `Algorithm`. Фабрика `ComputerPlayerFabric` создаёт нужную реализацию алгоритма по типу и уровню сложности в соответствии с предпочтениями пользователя приложения.

Алгоритмы (`ru.ac.uniyar.model.algorithms`) описаны в главе 2. Каждый из них реализует интерфейс `Algorithm`. Класс `AlgorithmReport` является неизменяемой записью с результатами последнего хода: выбранный ход, его оценка, достигнутая глубина поиска, количество рассмотренных узлов, количество отсечений ветвей дерева игры, время в миллисекундах, затраченное на анализ хода и его текстовое пояснение. Класс `EvaluationFeatures` хранит вектор признаков состояния; `EvaluationWeights` — веса; `EvaluationWeightsStore` управляет загрузкой и сохранением весов для историчности.

Сервисный слой (`ru.ac.uniyar.service`) включает несколько компонентов. `BoardFactory` создаёт начальное состояние доски заданного размера. Класс `GameRules` проверяет корректность установки стены и конвертирует координаты поля на пользовательском интерфейсе в игровые координаты, понятные алгоритмам. `GameProcessor` — аннотированный `@SessionScope` Spring-компонент, управляющий всем жизненным циклом партии: запуск (`startNewGame`), выполнение ходов (`makeMove`, `makeComputerMove`), сбор образцов для обучения весов и получение подсказок от всех четырёх алгоритмов в случае запроса таковых от пользователя приложения. `BoardAnalyzer` формирует текстовые пояснения к ходам ИИ. `TournamentService` позволяет запускать автоматические серии партий для турнира и сравнения алгоритмов. `SelfPlayTrainingService` отвечает за обучение весов параметров функции оценки: проводит партии между алгоритмами, собирает объекты `EvaluationLearningSample`, передаёт их в `EvaluationWeightsStore` и возвращает результат обучения в виде объекта `SelfPlayTrainingResult`.

Интерфейсный слой (`ru.ac.uniyar.ui`) реализован через библиотечные компоненты `Vaadin`. `StartPageController` отображает стартовый экран с выбором режима и параметров. `GamePageController` реализует основной экран игры: отрисовку игровой доски в виде сетки, историю ходов (`GameHistoryPanel`), кноп-

ки управления. `TournamentPageController`, `AlgorithmComparisonPageController` и `SelfPlayTrainingPageController` реализуют отображение режимов турнира, сравнения алгоритмов и обучения весов функции оценки.

3.2. Взаимодействие с приложением

При запуске игрового приложения «Коридор» пользователь попадает на стартовый экран, на котором настраиваются параметры предстоящей партии (рисунок 3).

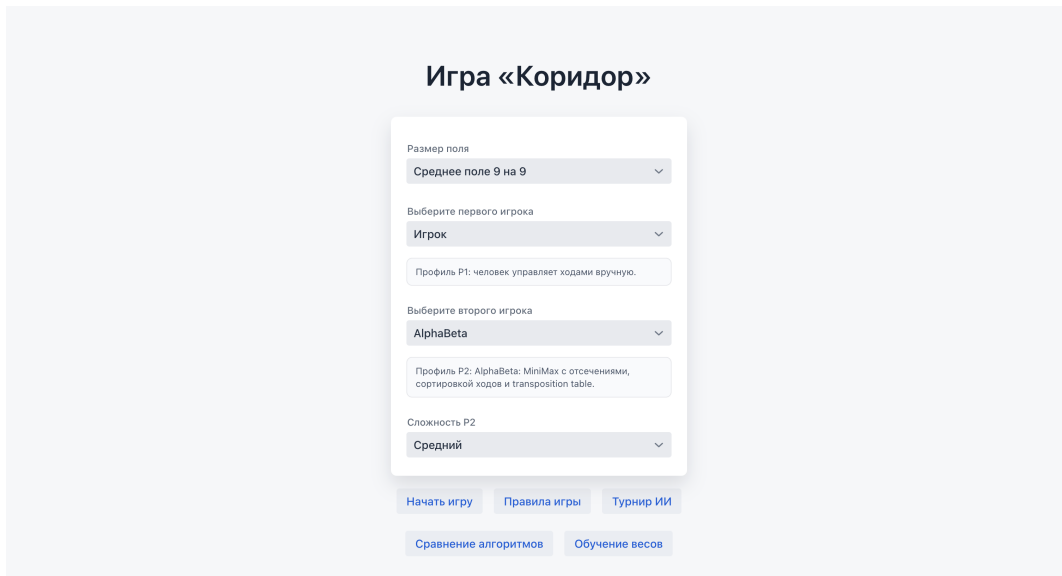


Рис. 3 — Стартовый экран выбора режима игры и алгоритмов

Доступны три варианта размера игрового поля: малое поле 7×7 с 8 стенами у каждого игрока, стандартное 9×9 с 10 стенами, большое 11×11 с 12 стенами. Размер поля влияет на все параметры алгоритмов: лимиты времени и максимальные глубины масштабируются пропорционально.

Первым игроком может управлять человек или один из четырёх алгоритмов программного соперника. Если выбран алгоритм, дополнительно настраивается его уровень сложности.

Второй игрок всегда управляется алгоритмом с настройкой его сложности. Карточки профилей отображают краткое описание выбранной стратегии.

На игровом экране пользователь видит несколько областей. Пример этого экрана приведён на рисунке 4.

Игровое поле отрисовывается в виде сетки: клетки чередуются с промежутками, в которых могут быть размещены стены, блокирующие передвижения между ними. Каждая клетка имеет размер 42–50 пикселей в зависимости от размера поля. Фишки отображаются цветными кружками разного цвета. Допустимые для перемещения клетки подсвечиваются зелёным при наведении. При наведении на щель между клетками предварительно подсвечивается потенциальная стена.

Перемещение фишки игрока выполняется кликом на целевую клетку. Если ход недопустим, появляется уведомление.

Установка стены выполняется кликом на промежуток между клетками. GameRules.extractWall определяет по координатам интерфейса начальную и конечную клетки перегородки. Затем GameRules.canPlaceWall копирует доску, устанавливает стену и проверяет достижимость целевых линий обоих игроков через обход в ширину.

В режиме «Игрок против ИИ» после хода пользователя автоматически запускается выбранный алгоритм. В режиме «ИИ против ИИ» каждый ход требует нажатия кнопки «Сделать ход ИИ», что позволяет наблюдать за партией пошагово, а не видеть лишь результат игры. Рядом с кнопкой отображается, чья очередь делать ход.

Кнопка «Подсказка хода» запускает все четыре алгоритма на текущей позиции и подсвечивает рекомендованные ими ходы разными цветами, каждый из которых соответствует одному из алгоритмов. Пример отображения подсказок приведён на рисунке 4.

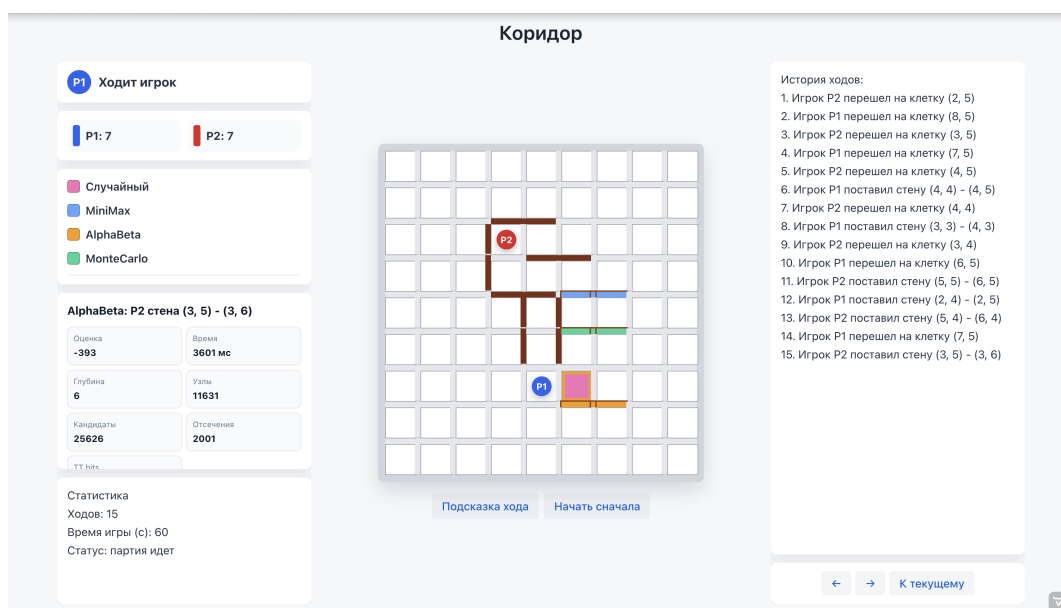


Рис. 4 — Отображение подсказок от алгоритмов на игровом поле

Панель информации отображает: чья очередь хода, количество оставшихся стен у каждого игрока, соответствие цветов и алгоритмов, а также отчёт о последнем ходе ИИ. Помимо прочего можно наблюдать за статистикой партии: числом ходов и затраченном времени.

С помощью кнопок «←», «→» и «К текущему» можно смотреть историю ходов. Она также дублируется текстовым списком в правой панели.

Турнирный режим позволяет выбрать два алгоритма, их уровни сложности, размер поля и число партий. Результаты выводятся в режиме реального времени: по завершении каждой партии обновляется счётчик побед. По завершении серии

выводится сводная таблица. Пример работы режима показан на рисунке 5.

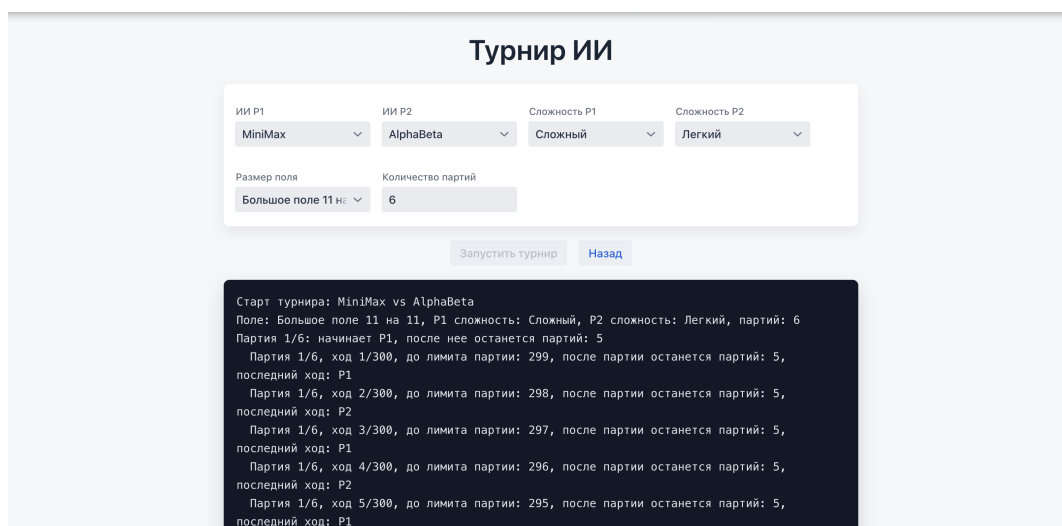


Рис. 5 — Экран турнирного режима ИИ

Режим сравнения алгоритмов автоматически проводит серию матчей между всеми попарными комбинациями четырёх алгоритмов и выводит сводную таблицу итогов с процентом побед, средним числом ходов, средним временем хода, средней глубиной поиска, числом узлов и отсечений. Интерфейс режима приведён на рисунке 6.

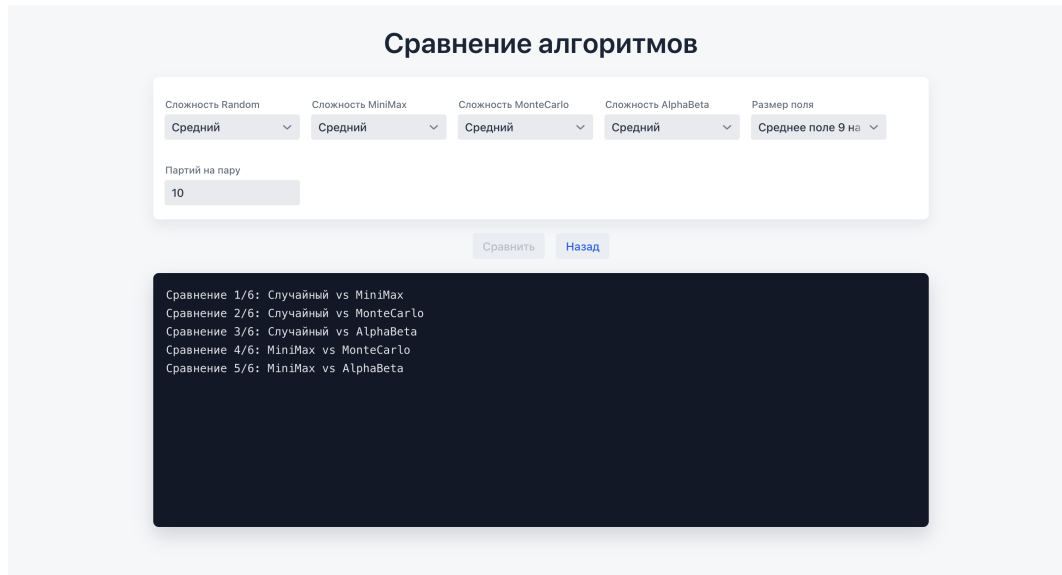


Рис. 6 — Экран сравнения алгоритмов

Режим обучения весов запускает серию партий между двумя выбранными алгоритмами. По завершении каждой партии сервис обучения обновляет веса функции оценки, сохраняет их в файл и выводит ход эксперимента в лог. Пример логов обучения показан на рисунке 7.

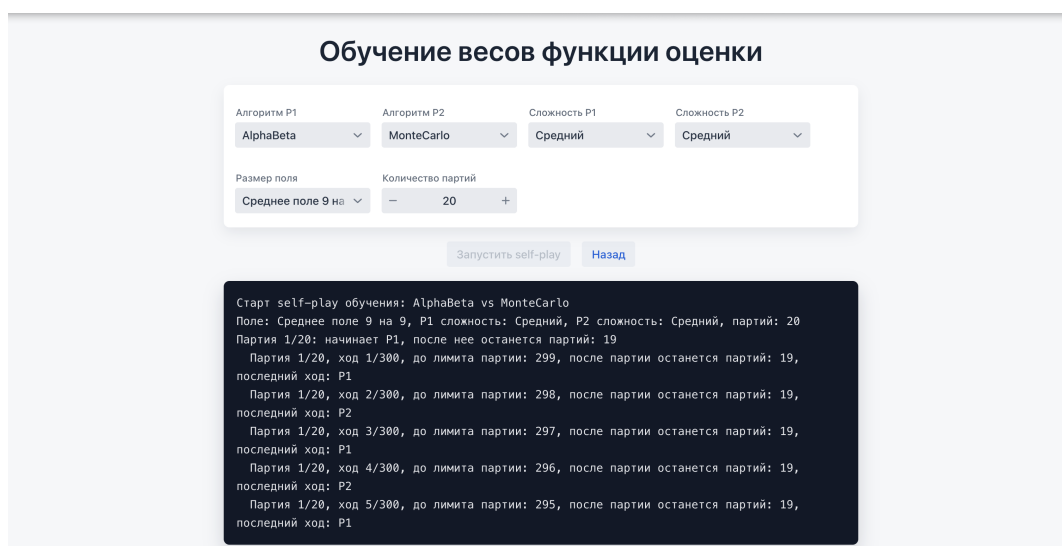


Рис. 7 — Экран self-play обучения весов функции оценки

4. Сравнение алгоритмов

4.1. Методика сравнения

Сравнение алгоритмов, как уже описывалось ранее, выполняется в отдельном режиме игрового приложения. Цель добавления режима — получить не единственный результат отдельной партии, а агрегированную оценку поведения стратегий на одинаковых условиях.

В эксперименте участвуют все применяемые в игровом приложении «Коридор» алгоритма. Они сравниваются попарно, количество различных пар равно:

$$C_4^2 = \frac{4 \cdot 3}{2} = 6.$$

Для каждой пары играется заданное пользователем число партий. Для компенсации преимущества первого хода роли игроков чередуются: часть партий алгоритм проводит за первого игрока, часть — за второго. Размер поля и уровень сложности каждого алгоритма задаются перед запуском эксперимента.

Важно отметить, что в режиме сравнения алгоритмов обучение весов функции оценки не выполняется.

4.2. Собираемые метрики

Для анализа недостаточно знать только победителя. Два алгоритма могут иметь близкую долю побед, но существенно отличаться по времени работы, глубине поиска или количеству просмотренных состояний игрового поля. Поэтому приложение собирает несколько параметров статистики.

Метрики глубины, узлов и отсечений особенно важны для сравнения алгоритмов `MiniMaxAlgorithm` и `AlphaBetaAlgorithm`. Оба алгоритма основаны на дереве игры, но альфа-бета должен просматривать меньше узлов при большей достижимой глубине за счёт отсечений. Для `MonteCarloAlgorithm` ключевыми являются число итераций и время хода, поскольку качество решения определяется количеством проведённых симуляций.

4.3. План эксперимента

Итоговый эксперимент проводится на большом поле 11×11 , поскольку такой размер увеличивает пространство возможных ходов, делает партии длиннее и сильнее проявляет различия между стратегиями. Для всех алгоритмов устанавливается уровень сложности `HARD`. Для каждой пары алгоритмов проводится 25

Таблица 4.1

Метрики сравнения алгоритмов

Метрика	Смысл
Количество партий	Общее число завершённых партий с участием алгоритма
Победы и поражения	Основная игровая результативность алгоритма
Доля побед	Доля побед алгоритма среди всех сыгранных им партий
Среднее число ходов	Косвенная характеристика стиля игры и сложности партий
Среднее время хода	Практическая вычислительная стоимость алгоритма
Средняя глубина	Насколько глубоко алгоритм успевает просмотреть дерево
Среднее число узлов	Объём реально просмотренного дерева поиска
Среднее число отсечений	Эффективность alpha-beta оптимизации

партий. Так как сравниваются четыре алгоритма, всего рассматривается шесть пар, а суммарное число партий равно 150.

Каждый алгоритм участвует в трёх попарных сравнениях, поэтому итоговая статистика для одного алгоритма агрегируется по 75 партиям. Роли первого и второго игрока чередуются между партиями; из-за нечётного числа партий в каждой паре один из алгоритмов начинает на одну партию больше, но при 25 партиях влияние этого перекося считается допустимым.

4.4. Форма представления результатов

Итоговая статистика представлена в двух формах. Первая форма — попарная таблица побед, показывающая, как конкретные алгоритмы играют друг против друга.

Таблица 4.2

Попарные результаты сравнения алгоритмов

Алгоритм P1	Алгоритм P2	Игры	Победы P1	Победы P2	Ср. ходов	Ср. мс	Ср. глуб.	Ср. узлы	Ср. отсеч.
Случайный	MiniMax	25	0	25	90,32	343,3	2,46	1270,5	0,0
Случайный	MonteCarlo	25	0	25	155,76	3162,8	54,96	174,6	0,0
Случайный	AlphaBeta	25	0	25	95,12	1308,7	3,87	2581,2	527,9
MiniMax	MonteCarlo	25	13	12	102,92	3563,4	56,23	1185,1	0,0
MiniMax	AlphaBeta	25	3	22	91,32	1958,8	5,30	4837,7	657,6
MonteCarlo	AlphaBeta	25	10	15	110,40	4522,9	57,92	2577,8	485,2

Вторая форма — агрегированная таблица по каждому алгоритму. Она удобнее для общего вывода, поскольку показывает не только победы, но и вычислительные затраты.

Сводная статистика алгоритмов

Алгоритм	Партий	Победы	Поражения	Доля побед	Очки/партия	Ср. ходов	Ср. мс	Ср. глуб.	Ср. узлы	Ср. отсеч.
Случайный	75	0	75	0,00%	0,00	113,73	1604,9	20,43	1342,1	176,0
MiniMax	75	41	34	54,67%	0,55	94,85	1955,2	21,33	2431,1	219,2
MonteCarlo	75	47	28	62,67%	0,63	123,03	3749,7	56,37	1312,5	161,7
AlphaBeta	75	62	13	82,67%	0,83	98,95	2596,8	22,36	3332,2	556,9

4.5. Анализ результатов

Фактические результаты подтверждают, что случайный алгоритм является базовой линией сравнения: на большом поле 11×11 он проиграл все 75 партий. Несмотря на использование функции оценки на высоком уровне сложности, случайный алгоритм не строит дерево ходов и не анализирует ответы соперника, поэтому уступает всем остальным стратегиям.

MiniMax выиграл 41 партию из 75 и показал долю побед 54,67%. Он уверенно обыгрывает случайный алгоритм и почти на равных играет с MonteCarlo в отдельной паре: 13 побед против 12. При этом MiniMax значительно проигрывает AlphaBeta — 3 победы против 22 в попарном сравнении, что отражает ограниченность полного перебора без отсечений.

MonteCarlo показал второй результат по общей доле побед — 62,67%. Среднее время хода у MonteCarlo оказалось максимальным — 3749,7 мс, как и партии с ним в среднем самые длинные.

Лучший итоговый результат показал алгоритм AlphaBeta: 62 победы из 75, доля побед 82,67% и 0,83 очка за партию. Он победил все остальные алгоритмы в попарных сравнениях, включая MonteCarlo со счётом 15:10. При этом AlphaBeta имеет наибольшее среднее число отсечений — 556,9. Это подтверждает, что преимущество алгоритма связано не только с самой функцией оценки, но и с механизмами оптимизации дерева поиска: сортировкой ходов и alpha-beta отсечениями.

Заключение

В ходе выполнения выпускной квалификационной работы была исследована настольная логическая игра «Коридор». Реализованы алгоритмы проверки корректности перемещения фишки с учётом всех правил прыжков через соперника и проверки корректности установки стены с помощью обхода графа в ширину. Реализован алгоритм поиска кратчайшего пути до целевой линии с восстановлением пути.

Разработана функция оценки позиции, учитывающая семь параметров. Реализован механизм коррекции весов функции оценки по результатам завершённых партий с сохранением в файл конфигурации между сессиями.

Реализованы четыре алгоритма программного соперника с тремя уровнями сложности каждый.

Разработано игровое приложение с режимами «Игрок против ИИ», «ИИ против ИИ», режимом подсказок одновременно от всех четырёх алгоритмов, просмотром истории партии, турнирным режимом и режимом сравнения алгоритмов.

Список литературы

- [1] BoardGameGeek. Games 100 [Электронный ресурс]. — Режим доступа: https://boardgamegeek.com/wiki/page/Games_100 (дата обращения: 02.04.2026).
- [2] Games, Hobby. Коридор. Правила игры [Электронный ресурс]. — Режим доступа: https://hobbygames.ru/download/rules/Quoridor_Rules.pdf (дата обращения: 03.04.2026).
- [3] Russell, S. Artificial Intelligence: A Modern Approach [Text] / S. Russell, P. Norvig. — Hoboken : Pearson, 2021.
- [4] Coulom, R. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search [Текст] / R. Coulom // Computers and Games. — 2006. — С. 72–83.
- [5] Vaadin. Vaadin Flow Documentation [Электронный ресурс]. — Режим доступа: <https://vaadin.com/docs> (дата обращения: 05.03.2026).

Реализация интерфейса алгоритма

Листинг А.1 — Интерфейс Algorithm

```

1 public interface Algorithm {
2     int WIN_SCORE = 100_000;
3
4     ComputerAlgorithmType getType();
5     Move getMove(Board board, ComputerPlayerHardnessLevel
        hardnessLevel, int playerId, int wallsLeft1, int
        wallsLeft2);
6
7     default AlgorithmReport getLastReport() {
8         return null;
9     }
10
11     default void setRecentPositions(List<Position>
        recentPositions) {}
12
13     default int movementPreference(Move move, Board board, int
        playerId, int size, List<Position> recentPositions) {
14         if (move.getMoveType() != MoveType.MOVE_PLAYER || move.
            getPlayerId() != playerId) {
15             return 0;
16         }
17
18         int score = 0;
19         Position current = getCurrentPosition(board, playerId);
20         Position target = move.getEndPosition();
21         int direction = playerId == 1 ? -1 : 1;
22         int rowDelta = target.row() - current.row();
23
24         if (rowDelta == direction) {
25             score += 45;
26         } else if (rowDelta == -direction) {
27             score -= 120;
28         }
29

```

```

30     for (int index = 0; index < recentPositions.size(); ++
        index) {
31         Position recent = recentPositions.get(index);
32         if (recent.equals(current)) {
33             continue;
34         }
35         if (target.equals(recent)) {
36             score -= index <= 1 ? 500 : 180;
37         }
38     }
39
40     int before = Math.abs(current.row() - getTargetRow(
        playerId, size));
41     int after = Math.abs(target.row() - getTargetRow(
        playerId, size));
42     score += (before - after) * 35;
43     return score;
44 }
45
46 default List<Move> getMoves(Board board, int playerId, int
    wallsLeft) {
47     List<Move> moves = new ArrayList<>();
48     Position pos = getCurrentPosition(board, playerId);
49
50     for (Position next : board.getAvailableMoves(pos)) {
51         moves.add(Move.movePlayer(playerId, next));
52     }
53
54     if (wallsLeft <= 0) {
55         return moves;
56     }
57
58     int size = (int) Math.sqrt(board.getTiles().size());
59     Set<String> used = new HashSet<>();
60     List<Position> enemyPath = getShortestPathCells(board,
        getCurrentPosition(board, 3 - playerId),
        getTargetRow(3 - playerId, size));
61     List<Position> myPath = getShortestPathCells(board,
        getCurrentPosition(board, playerId), getTargetRow(
        playerId, size));

```

```

62     List<Position> candidateCells = new ArrayList<>();
63     List<Position> enemyFront = enemyPath.subList(0, Math.
        min(12, enemyPath.size()));
64     List<Position> myFront = myPath.subList(0, Math.min(7,
        myPath.size()));
65     candidateCells.addAll(enemyFront);
66     candidateCells.addAll(myFront);
67     addNeighborhood(candidateCells, getCurrentPosition(
        board, 3 - playerId), size);
68     for (Position cell : enemyFront.subList(0, Math.min(5,
        enemyFront.size())) {
69         addNeighborhood(candidateCells, cell, size, 1);
70     }
71
72     int beforeEnemy = calculateShortestPath(board,
        getCurrentPosition(board, 3 - playerId),
        getTargetRow(3 - playerId, size));
73     int beforeMine = calculateShortestPath(board,
        getCurrentPosition(board, playerId), getTargetRow(
        playerId, size));
74     List<Move> wallMoves = new ArrayList<>();
75
76     for (Position cell : candidateCells) {
77         int i = cell.row();
78         int j = cell.col();
79
80         if (i < size - 1) {
81             Position start = new Position(i, j);
82             Position end = new Position(i + 1, j);
83             String wallKey = start + "-" + end;
84
85             if (used.add(wallKey) && isWallPlaceable(board,
                start, end) && isValidWall(board, start,
                end)) {
86                 if (isStrategicWall(board, start, end,
                    playerId, size, beforeEnemy, beforeMine)
                    ) {
87                     wallMoves.add(Move.placeWall(playerId,
                        start, end));
88                 }

```

```

89         }
90     }
91
92     if (j < size - 1) {
93         Position start = new Position(i, j);
94         Position end = new Position(i, j + 1);
95         String wallKey = start + "-" + end;
96
97         if (used.add(wallKey) && isWallPlaceable(board,
98             start, end) && isValidWall(board, start,
99             end)) {
100             if (isStrategicWall(board, start, end,
101                 playerId, size, beforeEnemy, beforeMine)
102                 ) {
103                 wallMoves.add(Move.placeWall(playerId,
104                     start, end));
105             }
106         }
107     }
108
109     wallMoves.sort((a, b) -> Integer.compare(wallImpact(
110         board, b, playerId, size), wallImpact(board, a,
111         playerId, size)));
112     moves.addAll(wallMoves);
113     return moves;
114 }
115
116 default void addNeighborhood(List<Position> cells, Position
117     center, int size) {
118     addNeighborhood(cells, center, size, 2);
119 }
120
121 default void addNeighborhood(List<Position> cells, Position
122     center, int size, int radius) {
123     for (int di = -radius; di <= radius; ++di) {
124         for (int dj = -radius; dj <= radius; ++dj) {
125             int nextRow = center.row() + di;
126             int nextCol = center.col() + dj;
127             if (nextRow >= 0 && nextCol >= 0 && nextRow <
128                 size && nextCol < size) {

```

```

119             cells.add(new Position(nextRow, nextCol));
120         }
121     }
122 }
123 }
124
125 default boolean isStrategicWall(Board board, Position start
    , Position end, int playerId, int size, int beforeEnemy,
    int beforeMine) {
126     Board copy = board.copy();
127     copy.placeWall(start, end);
128
129     int afterEnemy = calculateShortestPath(copy,
        getCurrentPosition(copy, 3 - playerId), getTargetRow
        (3 - playerId, size));
130     int afterMine = calculateShortestPath(copy,
        getCurrentPosition(copy, playerId), getTargetRow(
        playerId, size));
131
132     int enemyGain = afterEnemy - beforeEnemy;
133     int myCost = Math.max(0, afterMine - beforeMine);
134
135     if (enemyGain >= 2) {
136         return true;
137     }
138     if (beforeEnemy <= 3 && enemyGain > 0) {
139         return true;
140     }
141     return enemyGain > 0 && enemyGain >= myCost;
142 }
143
144 default boolean isWallPlaceable(Board board, Position start
    , Position end) {
145     try {
146         Board copy = board.copy();
147         copy.placeWall(start, end);
148         return true;
149     } catch (Exception e) {
150         return false;
151     }

```

```

152     }
153
154     default List<Position> getShortestPathCells(Board board,
        Position start, int targetRow) {
155         Map<Position, Position> parent = new HashMap<>();
156         Queue<Position> queue = new ArrayDeque<>();
157         Set<Position> visited = new HashSet<>();
158
159         queue.add(start);
160         visited.add(start);
161
162         Position end = null;
163         while (!queue.isEmpty()) {
164             Position curr = queue.poll();
165
166             if (curr.row() == targetRow) {
167                 end = curr;
168                 break;
169             }
170
171             for (Position next : board.getPathNeighbors(curr))
172                 {
173                     if (visited.add(next)) {
174                         parent.put(next, curr);
175                         queue.add(next);
176                     }
177                 }
178
179             List<Position> path = new ArrayList<>();
180             while (end != null) {
181                 path.add(end);
182                 end = parent.get(end);
183             }
184             Collections.reverse(path);
185             return path;
186         }
187
188     default boolean isValidWall(Board board, Position start,
        Position end) {

```

```

189         try {
190             Board copy = board.copy();
191             copy.placeWall(start, end);
192
193             int size = (int) Math.sqrt(board.getTiles().size())
194                 ;
195
196             return hasPath(copy, copy.getPositionOfPlayer1(),
197                 0) && hasPath(copy, copy.getPositionOfPlayer2(),
198                 size - 1);
199         } catch (Exception e) {
200             return false;
201         }
202     }
203
204     private boolean hasPath(Board board, Position start, int
205         targetRow) {
206         Set<Position> visited = new HashSet<>();
207         Queue<Position> queue = new ArrayDeque<>();
208
209         queue.add(start);
210         visited.add(start);
211
212         while (!queue.isEmpty()) {
213             Position curr = queue.poll();
214
215             if (curr.row() == targetRow) {
216                 return true;
217             }
218
219             for (Position next : board.getPathNeighbors(curr))
220             {
221                 if (visited.add(next)) {
222                     queue.add(next);
223                 }
224             }
225         }
226         return false;
227     }

```

```

224 default void applyMove(Board board, Move move) {
225     if (move.getMoveType() == MoveType.MOVE_PLAYER) {
226         if (move.getPlayerId() == 1) {
227             board.setPositionOfPlayer1(move.getEndPosition
228                 ());
229         } else {
230             board.setPositionOfPlayer2(move.getEndPosition
231                 ());
232         }
233     } else {
234         board.placeWall(move.getStartPosition(), move.
235             getEndPosition());
236     }
237 }
238
239 default Position getCurrentPosition(Board board, int
240     playerId) {
241     return playerId == 1 ? board.getPositionOfPlayer1() :
242         board.getPositionOfPlayer2();
243 }
244
245 default int getTargetRow(int playerId, int size) {
246     return playerId == 1 ? 0 : size - 1;
247 }
248
249 default boolean isWin(Board board, int playerId, int size)
250 {
251     return getCurrentPosition(board, playerId).row() ==
252         getTargetRow(playerId, size);
253 }
254
255 default Integer terminalScore(Board board, int rootPlayerId
256     , int size, int depth) {
257     if (isWin(board, rootPlayerId, size)) {
258         return WIN_SCORE + depth;
259     }
260     if (isWin(board, 3 - rootPlayerId, size)) {
261         return -WIN_SCORE - depth;
262     }
263     return null;

```



```

256     }
257
258     default Move findEndgameMove(Board board, int playerId, int
        wallsLeft) {
259         int size = (int) Math.sqrt(board.getTiles().size());
260         Position current = getCurrentPosition(board, playerId);
261         for (Position next : board.getAvailableMoves(current))
            {
262             if (next.row() == getTargetRow(playerId, size)) {
263                 return Move.movePlayer(playerId, next);
264             }
265         }
266
267         if (wallsLeft <= 0) {
268             return null;
269         }
270
271         int enemy = 3 - playerId;
272         int enemyDistance = calculateShortestPath(board,
            getCurrentPosition(board, enemy), getTargetRow(enemy
            , size));
273         if (enemyDistance > 2) {
274             return null;
275         }
276
277         Move bestWall = null;
278         int bestImpact = 0;
279         for (Move move : getMoves(board, playerId, wallsLeft))
            {
280             if (move.getMoveType() != MoveType.PLACE_WALL) {
281                 continue;
282             }
283             int impact = wallImpact(board, move, playerId, size
                );
284             if (impact > bestImpact) {
285                 bestImpact = impact;
286                 bestWall = move;
287             }
288         }
289         return bestImpact > 0 ? bestWall : null;

```

```

290     }
291
292     default boolean isNoisyPosition(Board board, int playerId,
293         int size) {
294         int myDist = calculateShortestPath(board,
295             getCurrentPosition(board, playerId), getTargetRow(
296                 playerId, size));
297         int enemyDist = calculateShortestPath(board,
298             getCurrentPosition(board, 3 - playerId),
299             getTargetRow(3 - playerId, size));
300         return myDist <= 2 || enemyDist <= 2;
301     }
302
303     default List<Move> getQuiescenceMoves(Board board, int
304         currentPlayer, int wallsLeft, int ignoredRootPlayerId,
305         int size) {
306         List<Move> moves = new ArrayList<>();
307         Position current = getCurrentPosition(board,
308             currentPlayer);
309         for (Position next : board.getAvailableMoves(current))
310         {
311             if (next.row() == getTargetRow(currentPlayer, size)
312                 ) {
313                 moves.add(Move.movePlayer(currentPlayer, next))
314                 ;
315             }
316         }
317
318         if (wallsLeft > 0) {
319             List<Move> tacticalWalls = getMoves(board,
320                 currentPlayer, wallsLeft).stream()
321                 .filter(move -> move.getMoveType() ==
322                     MoveType.PLACE_WALL)
323                 .sorted((a, b) -> Integer.compare(
324                     wallImpact(board, b, currentPlayer,
325                         size),
326                     wallImpact(board, a, currentPlayer,
327                         size)
328                 ))
329                 .limit(8)

```

```

315         .toList();
316         moves.addAll(tacticalWalls);
317     }
318     return moves;
319 }
320
321 default int wallImpact(Board board, Move move, int playerId
    , int size) {
322     int enemy = 3 - playerId;
323     int beforeEnemy = calculateShortestPath(board,
        getCurrentPosition(board, enemy), getTargetRow(enemy
        , size));
324     int beforeMine = calculateShortestPath(board,
        getCurrentPosition(board, playerId), getTargetRow(
        playerId, size));
325     Board copy = board.copy();
326     applyMove(copy, move);
327     int afterEnemy = calculateShortestPath(copy,
        getCurrentPosition(copy, enemy), getTargetRow(enemy,
        size));
328     int afterMine = calculateShortestPath(copy,
        getCurrentPosition(copy, playerId), getTargetRow(
        playerId, size));
329     return (afterEnemy - beforeEnemy) * 100 - Math.max(0,
        afterMine - beforeMine) * 45;
330 }
331
332 default int evaluate(Board board, int playerId, int size,
    int wallsLeft1, int wallsLeft2) {
333     Integer terminal = terminalScore(board, playerId, size,
        0);
334     if (terminal != null) {
335         return terminal;
336     }
337
338     return evaluationFeatures(board, playerId, size,
        wallsLeft1, wallsLeft2).score(EvaluationWeightsStore
        .current());
339 }
340

```

```

341  default EvaluationFeatures evaluationFeatures(Board board,
        int playerId, int size, int wallsLeft1, int wallsLeft2)
        {
342      int myDist = calculateShortestPath(board,
            getCurrentPosition(board, playerId), getTargetRow(
                playerId, size));
343      int enemyDist = calculateShortestPath(board,
            getCurrentPosition(board, 3 - playerId),
            getTargetRow(3 - playerId, size));
344
345      int myWalls = playerId == 1 ? wallsLeft1 : wallsLeft2;
346      int enemyWalls = playerId == 1 ? wallsLeft2 :
            wallsLeft1;
347
348      int myRow = getCurrentPosition(board, playerId).row();
349      int enemyRow = getCurrentPosition(board, 3 - playerId).
            row();
350      int myProgress = playerId == 1 ? size - 1 - myRow :
            myRow;
351      int enemyProgress = playerId == 1 ? enemyRow : size - 1
            - enemyRow;
352
353      return new EvaluationFeatures(
354          enemyDist - myDist,
355          board.getAvailableMoves(getCurrentPosition(
            board, playerId)).size(),
356          -board.getAvailableMoves(getCurrentPosition(
            board, 3 - playerId)).size(),
357          myWalls - enemyWalls,
358          myProgress - enemyProgress,
359          endgameFeature(myDist),
360          -endgameFeature(enemyDist)
361      );
362  }
363
364  default EvaluationLearningSample buildLearningSample(Board
        before, Move move, int playerId, int size, int
        wallsLeft1, int wallsLeft2) {
365      if (move == null) {
366          return null;

```

```

367     }
368
369     EvaluationFeatures beforeFeatures = evaluationFeatures(
        before, playerId, size, wallsLeft1, wallsLeft2);
370     Board after = before.copy();
371     applyMove(after, move);
372
373     int newWallsLeft1 = wallsLeft1;
374     int newWallsLeft2 = wallsLeft2;
375     if (move.getMoveType() == MoveType.PLACE_WALL) {
376         if (move.getPlayerId() == 1) {
377             --newWallsLeft1;
378         } else {
379             --newWallsLeft2;
380         }
381     }
382
383     EvaluationFeatures afterFeatures = evaluationFeatures(
        after, playerId, size, newWallsLeft1, newWallsLeft2)
        ;
384     return new EvaluationLearningSample(playerId,
        beforeFeatures, afterFeatures);
385 }
386
387 default int endgameFeature(int myDist) {
388     if (myDist <= 1) {
389         return 10;
390     }
391     if (myDist == 2) {
392         return 4;
393     }
394     if (myDist == 3) {
395         return 1;
396     }
397     return 0;
398 }
399
400 default int calculateShortestPath(Board board, Position
    start, int targetRow) {
401     Set<Position> visited = new HashSet<>();

```

```

402     Queue<Position> queue = new ArrayDeque<>();
403
404     queue.add(start);
405     visited.add(start);
406
407     int depth = 0;
408
409     while (!queue.isEmpty()) {
410         for (int k = queue.size(); k > 0; --k) {
411             Position curr = queue.poll();
412
413             if (curr.row() == targetRow) return depth;
414
415             for (Position next : board.getPathNeighbors(
416                 curr)) {
417                 if (visited.add(next)) {
418                     queue.add(next);
419                 }
420             }
421             depth++;
422         }
423         return 1000;
424     }
425
426     default long getTimeLimit(ComputerPlayerHardnessLevel level
427         , int size) {
428         int multiplier = Math.max(1, size / GameSize.NORMAL.
429             getAmountOfTilesPerSide());
430         return switch (level) {
431             case EASY -> 700L * multiplier;
432             case MEDIUM -> 1600L * multiplier;
433             case HARD -> 2800L * multiplier;
434         };
435     }
436
437     default String boardKey(Board board) {
438         StringBuilder key = new StringBuilder();
439         key.append(board.getPositionOfPlayer1()).append('/').
440             append(board.getPositionOfPlayer2()).append('/');

```

```

438
439     board.getTiles().entrySet().stream()
440         .sorted(Comparator
441             .comparingInt((Map.Entry<Position, ?>
442                 entry) -> entry.getKey().row())
443             .thenComparingInt(entry -> entry.getKey
444                 ().col()))
445         .forEach(entry -> {
446             key.append(entry.getKey());
447             key.append(entry.getValue().
448                 isLeftMovementAvailable() ? '1' : '0');
449             key.append(entry.getValue().
450                 isForwardMovementAvailable() ? '1' : '0'
451                 );
452             key.append(entry.getValue().
453                 isRightMovementAvailable() ? '1' : '0');
454             key.append(entry.getValue().
455                 isBackwardsMovementAvailable() ? '1' : '
456                 0');
457         });
458
459     return key.toString();
460 }
461 }

```

Реализация случайного алгоритма

Листинг Б.1 — Класс RandomAlgorithm

```

1 public class RandomAlgorithm implements Algorithm {
2     private static final Random random = new Random();
3     private AlgorithmReport lastReport;
4     private List<Position> recentPositions = List.of();
5
6     @Override
7     public ComputerAlgorithmType getType() {
8         return ComputerAlgorithmType.RANDOM;
9     }
10
11    @Override
12    public AlgorithmReport getLastReport() {
13        return lastReport;
14    }
15
16    @Override
17    public void setRecentPositions(List<Position>
18        recentPositions) {
19        this.recentPositions = recentPositions == null ? List.
20            of() : List.copyOf(recentPositions);
21    }
22
23    @Override
24    public Move getMove(Board board,
25        ComputerPlayerHardnessLevel hardnessLevel, int playerId,
26        int wallsLeft1, int wallsLeft2) {
27        long startedAt = System.currentTimeMillis();
28        int amountOfWallsLeft = playerId == 1 ? wallsLeft1 :
29            wallsLeft2;
30        List<Move> moves = getMoves(board, playerId,
31            amountOfWallsLeft);
32
33        if (moves.isEmpty()) return null;
34    }
35 }
```



```

29     Move tacticalMove = findEndgameMove(board, playerId,
        amountOfWallsLeft);
30     if (tacticalMove != null) {
31         lastReport = new AlgorithmReport(getType().
            getDescription(), tacticalMove,
            evaluateAfterMove(board, tacticalMove, playerId,
                wallsLeft1, wallsLeft2), 0, 1, 1, 0, 0, System.
                currentTimeMillis() - startedAt, "Случайный_алго
                ритм_применил_эндшпильное_правило_перед_случайны
                м_выбором");
32         return tacticalMove;
33     }
34
35     Move result;
36     switch (hardnessLevel) {
37         case MEDIUM -> {
38             moves.sort((a, b) -> Integer.compare(
                evaluateAfterMove(board, b, playerId,
                    wallsLeft1, wallsLeft2), evaluateAfterMove(
                    board, a, playerId, wallsLeft1, wallsLeft2))
                );
39             int half = moves.size() / 2 + 1;
40             result = moves.get(random.nextInt(half));
41         }
42         case HARD -> {
43             moves.sort((a, b) -> Integer.compare(
                evaluateAfterMove(board, b, playerId,
                    wallsLeft1, wallsLeft2), evaluateAfterMove(
                    board, a, playerId, wallsLeft1, wallsLeft2))
                );
44             int top = Math.min(3, moves.size());
45             result = moves.get(random.nextInt(top));
46         }
47         default -> result = moves.get(random.nextInt(moves.
            size()));
48     }
49     lastReport = new AlgorithmReport(getType().
        getDescription(), result, evaluateAfterMove(board,
        result, playerId, wallsLeft1, wallsLeft2), 1, moves.
        size(), moves.size(), 0, 0, System.currentTimeMillis

```

```

        () - startedAt, describeHardness(hardnessLevel));
50     return result;
51 }
52
53 private String describeHardness(ComputerPlayerHardnessLevel
    hardnessLevel) {
54     return switch (hardnessLevel) {
55         case EASY -> "Случайный_выбор_среди_всех_допустимых
            _ходов";
56         case MEDIUM -> "Случайный_выбор_из_верхней_половины
            _ходов_по_функции_оценки";
57         case HARD -> "Случайный_выбор_из_трех_лучших_ходов_
            по_функции_оценки";
58     };
59 }
60
61 private int evaluateAfterMove(Board board, Move move, int
    playerId, int wallsLeft1, int wallsLeft2) {
62     Board copy = board.copy();
63     applyMove(copy, move);
64     int size = (int) Math.sqrt(copy.getTiles().size());
65     int newWallsLeft1 = wallsLeft1;
66     int newWallsLeft2 = wallsLeft2;
67     if (move.getMoveType() == MoveType.PLACE_WALL) {
68         if (move.getPlayerId() == 1) {
69             --newWallsLeft1;
70         } else {
71             --newWallsLeft2;
72         }
73     }
74     return evaluate(copy, playerId, size, newWallsLeft1,
        newWallsLeft2) + movementPreference(move, board,
        playerId, size, recentPositions);
75 }
76 }

```

Реализация алгоритма MiniMax

Листинг В.1 — Класс MinimaxAlgorithm

```

1 public class MinimaxAlgorithm implements Algorithm {
2     private final Map<String, Integer> cache = new HashMap<>();
3     private AlgorithmReport lastReport;
4     private long nodesVisited;
5     private long consideredMoves;
6     private List<Position> recentPositions = List.of();
7
8     @Override
9     public ComputerAlgorithmType getType() {
10         return ComputerAlgorithmType.MINIMAX;
11     }
12
13     @Override
14     public AlgorithmReport getLastReport() {
15         return lastReport;
16     }
17
18     @Override
19     public void setRecentPositions(List<Position>
20         recentPositions) {
21         this.recentPositions = recentPositions == null ? List.
22             of() : List.copyOf(recentPositions);
23     }
24
25     @Override
26     public Move getMove(Board board,
27         ComputerPlayerHardnessLevel hardnessLevel, int playerId,
28         int wallsLeft1, int wallsLeft2) {
29         long startedAt = System.currentTimeMillis();
30         int size = (int) Math.sqrt(board.getTiles().size());
31         long timeLimit = getTimeLimit(hardnessLevel, size);
32         long endTime = System.currentTimeMillis() + timeLimit;
33         int maxDepth = getMaxDepth(hardnessLevel, size);

```

```

31     cache.clear();
32     nodesVisited = 0;
33     consideredMoves = 0;
34
35     int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
36     Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
37     if (tacticalMove != null) {
38         int tacticalScore = scoreMoveAfterApply(board,
            tacticalMove, playerId, size, wallsLeft1,
            wallsLeft2);
39         lastReport = new AlgorithmReport(getType().
            getDescription(), tacticalMove, tacticalScore,
            0, nodesVisited, 1, 0, 0, System.
                currentTimeMillis() - startedAt, "MiniMax_примен
ил_эндшпильное_правило:_немедленная_победа_или_с
рочная_блокировка");
40         return tacticalMove;
41     }
42
43     Move best = null;
44     int bestScore = 0;
45     int reachedDepth = 0;
46     for (int depth = 1; depth <= maxDepth; depth++) {
47         if (System.currentTimeMillis() > endTime) {
48             break;
49         }
50         SearchResult result = search(board, depth, playerId
            , size, wallsLeft1, wallsLeft2, endTime);
51         if (result.move() != null) {
52             best = result.move();
53             bestScore = result.score();
54             reachedDepth = depth;
55         }
56     }
57     lastReport = new AlgorithmReport(getType().
        getDescription(), best, bestScore, reachedDepth,
        nodesVisited, consideredMoves, 0, 0, System.
            currentTimeMillis() - startedAt, "MiniMax_перебрал_д

```

```

        ерево_без_alpha-beta_отсечений;_лимит_времени:_" +
        timeLimit + "_мс");
58     return best;
59 }
60
61 private int getMaxDepth(ComputerPlayerHardnessLevel level,
    int size) {
62     boolean smallBoard = size <= GameSize.SMALL.
        getAmountOfTilesPerSide();
63     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
64     return switch (level) {
65         case EASY -> 3;
66         case MEDIUM -> smallBoard ? 5 : 4;
67         case HARD -> largeBoard ? 4 : 5;
68     };
69 }
70
71 @Override
72 public long getTimeLimit(ComputerPlayerHardnessLevel level,
    int size) {
73     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
74     return switch (level) {
75         case EASY -> largeBoard ? 900L : 700L;
76         case MEDIUM -> largeBoard ? 2200L : 1800L;
77         case HARD -> largeBoard ? 4200L : 3600L;
78     };
79 }
80
81 private SearchResult search(Board board, int depth, int
    playerId, int size, int wallsLeft1, int wallsLeft2, long
    endTime) {
82     int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
83     List<Move> moves = orderedLimitedMoves(board, getMoves(
        board, playerId, currentWalls), playerId, size,
        wallsLeft1, wallsLeft2);
84     consideredMoves += moves.size();
85

```

```

86         Move bestMove = null;
87         int bestValue = Integer.MIN_VALUE;
88         for (Move move : moves) {
89             if (System.currentTimeMillis() > endTime) {
90                 return new SearchResult(bestMove, bestValue);
91             }
92
93             Board copy = board.copy();
94             applyMove(copy, move);
95
96             int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
                wallsLeft2;
97             if (move.getMoveType() == MoveType.PLACE_WALL) {
98                 if (playerId == 1) {
99                     --newWallsLeft1;
100                 } else {
101                     --newWallsLeft2;
102                 }
103             }
104
105             int value = minimax(copy, depth - 1, false,
106                 playerId, size, newWallsLeft1,
107                 newWallsLeft2, endTime);
108             value += movementPreference(move, board, playerId,
109                 size, recentPositions);
110             if (value > bestValue) {
111                 bestValue = value;
112                 bestMove = move;
113             }
114         }
115         return new SearchResult(bestMove, bestValue);
116     }
117
118     private int minimax(Board board, int depth, boolean
119         maximizing, int playerId, int size, int wallsLeft1, int
120         wallsLeft2, long endTime) {
121         nodesVisited++;
122         if (System.currentTimeMillis() > endTime) {
123             return evaluate(board, playerId, size, wallsLeft1,
124                 wallsLeft2);
125         }

```

```

120     }
121     String key = boardKey(board) + "/" + depth + "/" +
        maximizing + "/" + wallsLeft1 + "/" + wallsLeft2;
122     if (cache.containsKey(key)) {
123         return cache.get(key);
124     }
125
126     Integer terminal = terminalScore(board, playerId, size,
        depth);
127     if (terminal != null) {
128         cache.put(key, terminal);
129         return terminal;
130     }
131
132     if (depth == 0) {
133         int val = evaluate(board, playerId, size,
            wallsLeft1, wallsLeft2);
134         cache.put(key, val);
135         return val;
136     }
137
138     int current = maximizing ? playerId : (3 - playerId);
139     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
140     List<Move> moves = orderedLimitedMoves(board, getMoves(
        board, current, walls), playerId, size, wallsLeft1,
        wallsLeft2);
141     consideredMoves += moves.size();
142
143     int best = maximizing ? Integer.MIN_VALUE : Integer.
        MAX_VALUE;
144     for (Move move : moves) {
145         Board copy = board.copy();
146         applyMove(copy, move);
147
148         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
149         if (move.getMoveType() == MoveType.PLACE_WALL) {
150             if (current == 1) {
151                 --newWallsLeft1;
152             } else {

```

```

153         --newWallsLeft2;
154     }
155 }
156
157     int val = minimax(copy, depth - 1, !maximizing,
158         playerId, size, newWallsLeft1,
159         newWallsLeft2, endTime);
160
161     if (maximizing) {
162         best = Math.max(best, val);
163     } else {
164         best = Math.min(best, val);
165     }
166 }
167 cache.put(key, best);
168 return best;
169 }
170
171 private List<Move> orderedLimitedMoves(Board board, List<
172     Move> moves, int playerId, int size, int wallsLeft1, int
173     wallsLeft2) {
174     List<Move> ordered = new ArrayList<>(moves);
175     ordered.sort((a, b) -> Integer.compare(
176         scoreMoveForOrdering(board, b, playerId, size,
177         wallsLeft1, wallsLeft2),
178         scoreMoveForOrdering(board, a, playerId, size,
179         wallsLeft1, wallsLeft2)
180     ));
181     if (ordered.size() <= 16) {
182         return ordered;
183     }
184     return ordered.subList(0, 16);
185 }
186
187 private int scoreMoveForOrdering(Board board, Move move,
188     int playerId, int size, int wallsLeft1, int wallsLeft2)
189 {
190     return scoreMoveAfterApply(board, move, playerId, size,
191         wallsLeft1, wallsLeft2) + movementPreference(move,
192         board, playerId, size, recentPositions);
193 }

```



```

184
185     private int scoreMoveAfterApply(Board board, Move move, int
        playerId, int size, int wallsLeft1, int wallsLeft2) {
186         Board copy = board.copy();
187         applyMove(copy, move);
188         int newWallsLeft1 = wallsLeft1;
189         int newWallsLeft2 = wallsLeft2;
190         if (move.getMoveType() == MoveType.PLACE_WALL) {
191             if (move.getPlayerId() == 1) {
192                 --newWallsLeft1;
193             } else {
194                 --newWallsLeft2;
195             }
196         }
197         return evaluate(copy, playerId, size, newWallsLeft1,
            newWallsLeft2);
198     }
199
200     private record SearchResult(Move move, int score) {}
201 }

```

Реализация алгоритма AlphaBeta

Листинг Г.1 — Класс AlphaBetaAlgorithm

```

1 public class AlphaBetaAlgorithm implements Algorithm {
2     private final Map<String, TranspositionEntry>
        transpositionTable = new HashMap<>();
3     private final Map<String, Integer> goodMoves = new HashMap
        <>();
4     private final Move[][] bestMoves = new Move[50][2];
5     private AlgorithmReport lastReport;
6     private long nodesVisited;
7     private long consideredMoves;
8     private long cutoffs;
9     private long tableHits;
10    private List<Position> recentPositions = List.of();
11
12    @Override
13    public ComputerAlgorithmType getType() {
14        return ComputerAlgorithmType.ALPHABETA;
15    }
16
17    @Override
18    public AlgorithmReport getLastReport() {
19        return lastReport;
20    }
21
22    @Override
23    public void setRecentPositions(List<Position>
        recentPositions) {
24        this.recentPositions = recentPositions == null ? List.
            of() : List.copyOf(recentPositions);
25    }
26
27    @Override
28    public Move getMove(Board board,
        ComputerPlayerHardnessLevel level, int playerId, int
        wallsLeft1, int wallsLeft2) {

```

```

29      long startedAt = System.currentTimeMillis();
30      int size = (int) Math.sqrt(board.getTiles().size());
31      int maxDepth = getMaxDepth(level, size);
32      long timeLimit = getTimeLimit(level, size);
33      long endTime = System.currentTimeMillis() + timeLimit;
34
35      transpositionTable.clear();
36      nodesVisited = 0;
37      consideredMoves = 0;
38      cutoffs = 0;
39      tableHits = 0;
40
41      int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
42      Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
43      if (tacticalMove != null) {
44          int tacticalScore = scoreMoveAfterApply(board,
            tacticalMove, playerId, size, wallsLeft1,
            wallsLeft2);
45          lastReport = new AlgorithmReport(getType().
            getDescription(), tacticalMove, tacticalScore,
            0, nodesVisited, 1, cutoffs, tableHits, System.
            currentTimeMillis() - startedAt, "AlphaBeta_прим
            енил_эндшпильное_правило:_немедленная_победа_или
            _срочная_блокировка");
46          return tacticalMove;
47      }
48
49      Move bestMove = null;
50      int bestScore = 0;
51      int reachedDepth = 0;
52      for (int depth = 1; depth <= maxDepth; ++depth) {
53          if (System.currentTimeMillis() > endTime) {
54              break;
55          }
56          SearchResult result = search(board, depth, playerId
            , size, wallsLeft1, wallsLeft2, endTime);
57          if (result.move() != null) {
58              bestMove = result.move();

```

```

59         bestScore = result.score();
60         reachedDepth = depth;
61     }
62 }
63 lastReport = new AlgorithmReport(getType().
    getDescription(), bestMove, bestScore, reachedDepth,
    nodesVisited, consideredMoves, cutoffs, tableHits,
    System.currentTimeMillis() - startedAt, "AlphaBeta_и
    использует_сортировку_ходов,_beta_<=_alpha_отсечения_
    и_transposition_table_с_depth-bound_flags;_попаданий_
    в_таблицу:_" + tableHits + ";_лимит_времени:_" +
    timeLimit + "_мс");
64     return bestMove;
65 }
66
67 private int getMaxDepth(ComputerPlayerHardnessLevel level,
    int size) {
68     boolean smallBoard = size <= GameSize.SMALL.
        getAmountOfTilesPerSide();
69     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
70     return switch (level) {
71         case EASY -> smallBoard ? 5 : 4;
72         case MEDIUM -> smallBoard ? 7 : 6;
73         case HARD -> largeBoard ? 7 : 8;
74     };
75 }
76
77 @Override
78 public long getTimeLimit(ComputerPlayerHardnessLevel level,
    int size) {
79     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
80     boolean smallBoard = size <= GameSize.SMALL.
        getAmountOfTilesPerSide();
81     return switch (level) {
82         case EASY -> smallBoard ? 1000L : 1300L;
83         case MEDIUM -> largeBoard ? 4800L : 3600L;
84         case HARD -> largeBoard ? 9500L : 7600L;
85     };

```

```

86     }
87
88     private SearchResult search(Board board, int depth, int
        playerId, int size, int wallsLeft1, int wallsLeft2, long
        endTime) {
89         int currentWalls = playerId == 1 ? wallsLeft1 :
            wallsLeft2;
90         List<Move> moves = new ArrayList<>(getMoves(board,
            playerId, currentWalls).stream()
91             .filter(move -> isRootMoveLegal(board, move,
                playerId, currentWalls)).toList());
92         orderMoves(moves, board, playerId, size, wallsLeft1,
            wallsLeft2, 0, null);
93         if (moves.size() > 36) {
94             moves = moves.subList(0, 36);
95         }
96         consideredMoves += moves.size();
97
98         Move bestMove = null;
99         int best = Integer.MIN_VALUE;
100        for (Move move : moves) {
101            if (System.currentTimeMillis() > endTime) {
102                break;
103            }
104            Board copy = board.copy();
105            applyMove(copy, move);
106
107            int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
                wallsLeft2;
108            if (move.getMoveType() == MoveType.PLACE_WALL) {
109                if (playerId == 1) {
110                    --newWallsLeft1;
111                } else {
112                    --newWallsLeft2;
113                }
114            }
115            int val = alphaBeta(copy, depth - 1, Integer.
                MIN_VALUE, Integer.MAX_VALUE, false, playerId,
                size, newWallsLeft1, newWallsLeft2, endTime, 1);
116            val += rootMovePreference(move, board, playerId,

```

```

        size);
117         if (val > best) {
118             best = val;
119             bestMove = move;
120         }
121     }
122     return new SearchResult(bestMove, best);
123 }
124
125 private int alphaBeta(Board board, int depth, int alpha,
    int beta, boolean max, int playerId, int size, int
    wallsLeft1, int wallsLeft2, long endTime, int ply) {
126     nodesVisited++;
127     if (System.currentTimeMillis() > endTime) {
128         return evaluate(board, playerId, size, wallsLeft1,
            wallsLeft2);
129     }
130     String key = boardKey(board) + "/" + max + "/" +
        wallsLeft1 + "/" + wallsLeft2;
131     int originalAlpha = alpha;
132     int originalBeta = beta;
133
134     TranspositionEntry cached = transpositionTable.get(key)
        ;
135     if (cached != null && cached.depth >= depth) {
136         if (cached.bound == Bound.EXACT) {
137             tableHits++;
138             return cached.score;
139         }
140         if (cached.bound == Bound.LOWER) {
141             alpha = Math.max(alpha, cached.score);
142         } else if (cached.bound == Bound.UPPER) {
143             beta = Math.min(beta, cached.score);
144         }
145         if (alpha >= beta) {
146             tableHits++;
147             return cached.score;
148         }
149     }
150

```

```

151     Integer terminal = terminalScore(board, playerId, size,
        depth);
152     if (terminal != null) {
153         transpositionTable.put(key, new TranspositionEntry(
            depth, terminal, Bound.EXACT, null));
154         return terminal;
155     }
156
157     if (depth == 0) {
158         if (isNoisyPosition(board, playerId, size)) {
159             int calmScore = quiescence(board, alpha, beta,
                max, playerId, size, wallsLeft1, wallsLeft2,
                endTime, 2);
160             transpositionTable.put(key, new
                TranspositionEntry(depth, calmScore, Bound.
                EXACT, null));
161             return calmScore;
162         }
163         int eval = evaluate(board, playerId, size,
            wallsLeft1, wallsLeft2);
164         transpositionTable.put(key, new TranspositionEntry(
            depth, eval, Bound.EXACT, null));
165         return eval;
166     }
167
168     int current = max ? playerId : 3 - playerId;
169     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
170
171     List<Move> moves = getMoves(board, current, walls);
172     Move tableBestMove = cached == null ? null : cached.
        bestMove;
173     orderMoves(moves, board, playerId, size, wallsLeft1,
        wallsLeft2, ply, tableBestMove);
174     if (moves.size() > 36) {
175         moves = moves.subList(0, 36);
176     }
177     consideredMoves += moves.size();
178
179     int best = max ? Integer.MIN_VALUE : Integer.MAX_VALUE;
180     Move bestMove = null;

```

```

181
182     for (Move move : moves) {
183         Board copy = board.copy();
184         applyMove(copy, move);
185
186         int newWallsLeft1 = wallsLeft1, newWallsLeft2 =
            wallsLeft2;
187         if (move.getMoveType() == MoveType.PLACE_WALL) {
188             if (current == 1) {
189                 --newWallsLeft1;
190             } else {
191                 --newWallsLeft2;
192             }
193         }
194
195         int val = alphaBeta(copy, depth - 1, alpha, beta, !
            max, playerId, size, newWallsLeft1,
            newWallsLeft2, endTime, ply + 1);
196
197         if (max) {
198             if (val > best) {
199                 best = val;
200                 bestMove = move;
201             }
202             alpha = Math.max(alpha, val);
203         } else {
204             if (val < best) {
205                 best = val;
206                 bestMove = move;
207             }
208             beta = Math.min(beta, val);
209         }
210
211         if (beta <= alpha) {
212             cutoffs++;
213             updateBestMoves(move, ply);
214             updateGoodMoves(move);
215             break;
216         }
217     }

```



```

218
219     Bound bound = Bound.EXACT;
220     if (best <= originalAlpha) {
221         bound = Bound.UPPER;
222     } else if (best >= originalBeta) {
223         bound = Bound.LOWER;
224     }
225     transpositionTable.put(key, new TranspositionEntry(
226         depth, best, bound, bestMove));
227     return best;
228 }
229
230 private int quiescence(Board board, int alpha, int beta,
231     boolean max, int playerId, int size, int wallsLeft1, int
232     wallsLeft2, long endTime, int extensionDepth) {
233     nodesVisited++;
234     int standPat = evaluate(board, playerId, size,
235         wallsLeft1, wallsLeft2);
236     if (extensionDepth == 0 || System.currentTimeMillis() >
237         endTime) {
238         return standPat;
239     }
240
241     if (max) {
242         if (standPat >= beta) {
243             return beta;
244         }
245         alpha = Math.max(alpha, standPat);
246     } else {
247         if (standPat <= alpha) {
248             return alpha;
249         }
250         beta = Math.min(beta, standPat);
251     }
252
253     int current = max ? playerId : 3 - playerId;
254     int walls = current == 1 ? wallsLeft1 : wallsLeft2;
255     List<Move> moves = getQuiescenceMoves(board, current,
256         walls, playerId, size);
257     if (moves.isEmpty()) {

```

```

252         return standPat;
253     }
254     orderMoves(moves, board, playerId, size, wallsLeft1,
                wallsLeft2, 0, null);
255
256     for (Move move : moves) {
257         if (System.currentTimeMillis() > endTime) {
258             break;
259         }
260         Board copy = board.copy();
261         applyMove(copy, move);
262
263         int newWallsLeft1 = wallsLeft1;
264         int newWallsLeft2 = wallsLeft2;
265         if (move.getMoveType() == MoveType.PLACE_WALL) {
266             if (current == 1) {
267                 --newWallsLeft1;
268             } else {
269                 --newWallsLeft2;
270             }
271         }
272
273         int value = quiescence(copy, alpha, beta, !max,
                               playerId, size, newWallsLeft1, newWallsLeft2,
                               endTime, extensionDepth - 1);
274         if (max) {
275             if (value >= beta) {
276                 cutoffs++;
277                 return beta;
278             }
279             alpha = Math.max(alpha, value);
280         } else {
281             if (value <= alpha) {
282                 cutoffs++;
283                 return alpha;
284             }
285             beta = Math.min(beta, value);
286         }
287     }
288     return max ? alpha : beta;

```

```

289     }
290
291     private void orderMoves(List<Move> moves, Board board, int
        playerId, int size, int wallsLeft1, int wallsLeft2, int
        ply, Move tableBestMove) {
292         moves.sort((a, b) -> Integer.compare(scoreMove(b, board
            , playerId, size, wallsLeft1, wallsLeft2, ply,
            tableBestMove), scoreMove(a, board, playerId, size,
            wallsLeft1, wallsLeft2, ply, tableBestMove)));
293     }
294
295     private int scoreMove(Move move, Board board, int playerId,
        int size, int wallsLeft1, int wallsLeft2, int ply, Move
        tableBestMove) {
296         int score = 0;
297
298         if (move.equals(tableBestMove)) {
299             score += 20000;
300         }
301
302         if (ply < bestMoves.length) {
303             if (bestMoves[ply][0] != null && move.equals(
                bestMoves[ply][0])) {
304                 score += 10000;
305             }
306             if (bestMoves[ply][1] != null && move.equals(
                bestMoves[ply][1])) {
307                 score += 8000;
308             }
309         }
310
311         score += goodMoves.getOrDefault(move.toString(), 0) *
            2;
312         score += rootMovePreference(move, board, playerId, size
            );
313         score += scoreMoveAfterApply(board, move, playerId,
            size, wallsLeft1, wallsLeft2);
314
315         return score;
316     }

```

```

317
318     private int scoreMoveAfterApply(Board board, Move move, int
        playerId, int size, int wallsLeft1, int wallsLeft2) {
319         Board copy = board.copy();
320         applyMove(copy, move);
321         int newWallsLeft1 = wallsLeft1;
322         int newWallsLeft2 = wallsLeft2;
323         if (move.getMoveType() == MoveType.PLACE_WALL) {
324             if (move.getPlayerId() == 1) {
325                 --newWallsLeft1;
326             } else {
327                 --newWallsLeft2;
328             }
329         }
330         return evaluate(copy, playerId, size, newWallsLeft1,
            newWallsLeft2);
331     }
332
333     private int rootMovePreference(Move move, Board board, int
        playerId, int size) {
334         return movementPreference(move, board, playerId, size,
            recentPositions);
335     }
336
337     private boolean isRootMoveLegal(Board board, Move move, int
        playerId, int wallsLeft) {
338         if (move.getPlayerId() != playerId) {
339             return false;
340         }
341         if (move.getMoveType() == MoveType.MOVE_PLAYER) {
342             return board.getAvailableMoves(getCurrentPosition(
                board, playerId)).contains(move.getEndPosition()
            );
343         }
344         if (wallsLeft <= 0) {
345             return false;
346         }
347         return isWallPlaceable(board, move.getStartPosition(),
            move.getEndPosition()) && isValidWall(board, move.
            getStartPosition(), move.getEndPosition());

```

```

348     }
349
350     private void updateBestMoves(Move move, int ply) {
351         if (ply >= bestMoves.length || move.equals(bestMoves[
352             ply][0])) {
353             return;
354         }
355         bestMoves[ply][1] = bestMoves[ply][0];
356         bestMoves[ply][0] = move;
357     }
358
359     private void updateGoodMoves(Move move) {
360         goodMoves.merge(move.toString(), 1, Integer::sum);
361     }
362
363     private record SearchResult(Move move, int score) {}
364
365     private enum Bound {
366         EXACT,
367         LOWER,
368         UPPER
369     }
370
371     private record TranspositionEntry(int depth, int score,
        Bound bound, Move bestMove) {}
372 }

```

Реализация алгоритма Monte Carlo

Листинг Д.1 — Класс MonteCarloAlgorithm

```

1 public class MonteCarloAlgorithm implements Algorithm {
2     private static final double C = 1.2;
3     private static final Random random = new Random();
4     private AlgorithmReport lastReport;
5     private List<Position> recentPositions = List.of();
6     private int rootPlayerId;
7
8     @Override
9     public ComputerAlgorithmType getType() {
10         return ComputerAlgorithmType.MONTECARLO;
11     }
12
13     @Override
14     public AlgorithmReport getLastReport() {
15         return lastReport;
16     }
17
18     @Override
19     public void setRecentPositions(List<Position>
20         recentPositions) {
21         this.recentPositions = recentPositions == null ? List.
22             of() : List.copyOf(recentPositions);
23     }
24
25     @Override
26     public Move getMove(Board board,
27         ComputerPlayerHardnessLevel hardnessLevel, int playerId,
28         int wallsLeft1, int wallsLeft2) {
29         long startedAt = System.currentTimeMillis();
30         int size = (int) Math.sqrt(board.getTiles().size());
31         long timeLimit = getTimeLimit(hardnessLevel, size);
32         long endTime = System.currentTimeMillis() + timeLimit;
33         int rolloutDepth = getRolloutDepth(hardnessLevel, size)
34             ;

```

```

30     long iterationBudget = getIterationBudget(hardnessLevel
        , size);
31
32     long iterations = 0;
33     rootPlayerId = playerId;
34     int currentWalls = playerId == 1 ? wallsLeft1 :
        wallsLeft2;
35     Move tacticalMove = findEndgameMove(board, playerId,
        currentWalls);
36     if (tacticalMove != null) {
37         lastReport = new AlgorithmReport(getType().
            getDescription(), tacticalMove, scoreMove(board,
                tacticalMove, playerId, wallsLeft1, wallsLeft2)
                , 0, 0, 1, 0, 0, System.currentTimeMillis() -
                startedAt, "MCTS_применил_эндшпильное_правило_до
                _запуска_симуляций");
38         return tacticalMove;
39     }
40
41     Node root = new Node(board.copy(), null, null, playerId
        , wallsLeft1, wallsLeft2);
42     while (System.currentTimeMillis() < endTime &&
        iterations < iterationBudget) {
43         Node node = select(root);
44         Node expanded = expand(node);
45         int winner = simulate(expanded, rolloutDepth,
            playerId, size);
46         backpropagate(expanded, winner, playerId);
47         iterations++;
48         if (root.visits > 300 && bestWinRate(root) > 0.97)
            {
49             break;
50         }
51     }
52
53     Node best = root.children.stream()
54         .max(Comparator.comparingDouble(this::
            robustChildScore))
55         .orElseThrow();
56     lastReport = new AlgorithmReport(getType().

```

```

        getDescription(), best.move, evaluate(best.board,
        playerId, size, best.walls1, best.walls2),
        rolloutDepth, iterations, root.children.size(), 0,
        0, System.currentTimeMillis() - startedAt, "MCTS:_вы
        бор_по_UCT,_rollout_с_epsilon-greedy_эвристикой,_нем
        едленными_победами_и_тактическими_стенами;_лимит_вре
        мени:_" + timeLimit + "_мс,_бюджет_итераций:_" +
        iterationBudget);
57     return best.move;
58 }
59
60 private int getRolloutDepth(ComputerPlayerHardnessLevel
    level, int size) {
61     return switch (level) {
62         case EASY -> size * 4;
63         case MEDIUM -> size * 7;
64         case HARD -> size * 10;
65     };
66 }
67
68 private long getIterationBudget(ComputerPlayerHardnessLevel
    level, int size) {
69     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
70     return switch (level) {
71         case EASY -> largeBoard ? 380L : 520L;
72         case MEDIUM -> largeBoard ? 1300L : 1700L;
73         case HARD -> largeBoard ? 2600L : 3400L;
74     };
75 }
76
77 @Override
78 public long getTimeLimit(ComputerPlayerHardnessLevel level,
    int size) {
79     boolean largeBoard = size > GameSize.NORMAL.
        getAmountOfTilesPerSide();
80     return switch (level) {
81         case EASY -> largeBoard ? 1200L : 950L;
82         case MEDIUM -> largeBoard ? 3800L : 2900L;
83         case HARD -> largeBoard ? 7600L : 6200L;

```



```

84         };
85     }
86
87     private Node select(Node node) {
88         while (!node.children.isEmpty() && node.untriedMoves.
            isEmpty()) {
89             node = node.children.stream().max(Comparator.
                comparing(this::uct)).orElseThrow();
90         }
91         return node;
92     }
93
94     private Node expand(Node node) {
95         if (node.untriedMoves.isEmpty() && !node.children.
            isEmpty()) {
96             return node;
97         }
98
99         if (node.untriedMoves.isEmpty()) {
100             node.untriedMoves.addAll(getMoves(node.board, node.
                player, node.getWalls()));
101             node.untriedMoves.sort((a, b) -> Integer.compare(
102                 scoreMove(node.board, b, node.player, node.
                    walls1, node.walls2),
103                 scoreMove(node.board, a, node.player, node.
                    walls1, node.walls2)
104             ));
105         }
106
107         if (node.untriedMoves.isEmpty()) {
108             return node;
109         }
110
111         Move move = node.untriedMoves.remove(0);
112         Board copy = node.board.copy();
113         applyMove(copy, move);
114
115         int wallsLeft1 = node.walls1;
116         int wallsLeft2 = node.walls2;
117

```

```

118         if (move.getMoveType() == MoveType.PLACE_WALL) {
119             if (node.player == 1) {
120                 --wallsLeft1;
121             } else {
122                 --wallsLeft2;
123             }
124         }
125         Node child = new Node(copy, node, move, 3 - node.player
126             , wallsLeft1, wallsLeft2);
127         node.children.add(child);
128         return child;
129     }
130     private int simulate(Node node, int maxSteps, int
131         rootPlayer, int size) {
132         Board board = node.board.copy();
133         int currentPlayer = node.player;
134         int wallsLeft1 = node.walls1;
135         int wallsLeft2 = node.walls2;
136
137         for (int step = 0; step < maxSteps; step++) {
138             if (isWin(board, 1, size)) {
139                 return 1;
140             }
141             if (isWin(board, 2, size)) {
142                 return 2;
143             }
144
145             int walls = currentPlayer == 1 ? wallsLeft1 :
146                 wallsLeft2;
147             List<Move> moves = getMoves(board, currentPlayer,
148                 walls);
149             if (moves.isEmpty()) {
150                 return 3 - currentPlayer;
151             }
152
153             Move best = pickRolloutMove(board, moves,
154                 currentPlayer, rootPlayer, size, wallsLeft1,
155                 wallsLeft2);

```

```

152         applyMove(board, best);
153
154         if (best.getMoveType() == MoveType.PLACE_WALL) {
155             if (currentPlayer == 1) {
156                 --wallsLeft1;
157             } else {
158                 --wallsLeft2;
159             }
160         }
161         currentPlayer = 3 - currentPlayer;
162     }
163
164     return evaluate(board, rootPlayer, size, wallsLeft1,
165                  wallsLeft2) > 0 ? rootPlayer : (3 - rootPlayer);
166 }
167
168 private void backpropagate(Node node, int winner, int
169                             playerId) {
170     while (node != null) {
171         ++node.visits;
172         if (winner == playerId) {
173             ++node.wins;
174         }
175         node = node.parent;
176     }
177 }
178
179 private double uct(Node n) {
180     if (n.visits == 0) {
181         return Double.MAX_VALUE;
182     }
183
184     double exploitation = n.wins / n.visits;
185     double exploration = C * Math.sqrt(Math.log(n.parent.
186                                         visits) / n.visits);
187     double heuristic = normalizedEvaluate(n.board, n.parent
188                                         .player, n.walls1, n.walls2);
189
190     return exploitation + exploration + heuristic;
191 }

```

```

188
189     private Move pickRolloutMove(Board board, List<Move> moves,
190         int currentPlayer, int rootPlayer, int size, int
191         wallsLeft1, int wallsLeft2) {
192         for (Move move : moves) {
193             if (move.getMoveType() == MoveType.MOVE_PLAYER
194                 && move.getEndPosition().row() ==
195                 getTargetRow(currentPlayer, size)) {
196                 return move;
197             }
198         }
199
200         int walls = currentPlayer == 1 ? wallsLeft1 :
201             wallsLeft2;
202         if (walls > 0 && calculateShortestPath(board,
203             getCurrentPosition(board, 3 - currentPlayer),
204             getTargetRow(3 - currentPlayer, size)) <= 2) {
205             Move tacticalWall = moves.stream()
206                 .filter(move -> move.getMoveType() ==
207                     MoveType.PLACE_WALL)
208                 .max(Comparator.comparingInt(move ->
209                     wallImpact(board, move, currentPlayer,
210                         size)))
211                 .orElse(null);
212             if (tacticalWall != null && wallImpact(board,
213                 tacticalWall, currentPlayer, size) > 0) {
214                 return tacticalWall;
215             }
216         }
217
218         if (random.nextDouble() < 0.22) {
219             return moves.get(random.nextInt(moves.size()));
220         }
221
222         int sampleSize = Math.min(18, moves.size());
223         List<Move> sampled = new ArrayList<>(sampleSize);
224         Set<Integer> usedIndexes = new HashSet<>();
225         while (sampled.size() < sampleSize) {
226             int index = random.nextInt(moves.size());
227             if (usedIndexes.add(index)) {

```

```

219         sampled.add(moves.get(index));
220     }
221 }
222
223     sampled.sort((a, b) -> Integer.compare(
224         scoreRolloutMove(board, b, currentPlayer,
225             rootPlayer, size, wallsLeft1, wallsLeft2),
226         scoreRolloutMove(board, a, currentPlayer,
227             rootPlayer, size, wallsLeft1, wallsLeft2)
228     ));
229     int top = Math.max(1, Math.min(4, sampled.size()));
230     return sampled.get(random.nextInt(top));
231 }
232
233 private int scoreRolloutMove(Board board, Move move, int
234     currentPlayer, int rootPlayer, int size, int wallsLeft1,
235     int wallsLeft2) {
236     Board tmp = board.copy();
237     applyMove(tmp, move);
238     int newWallsLeft1 = wallsLeft1;
239     int newWallsLeft2 = wallsLeft2;
240     if (move.getMoveType() == MoveType.PLACE_WALL) {
241         if (currentPlayer == 1) {
242             --newWallsLeft1;
243         } else {
244             --newWallsLeft2;
245         }
246     }
247
248     int score = evaluate(tmp, rootPlayer, size,
249         newWallsLeft1, newWallsLeft2);
250     if (currentPlayer != rootPlayer) {
251         score = -score;
252     }
253     if (currentPlayer == rootPlayer) {
254         score += movementPreference(move, board,
255             currentPlayer, size, recentPositions);
256     }
257     return score;
258 }

```

```

253
254     private int scoreMove(Board board, Move move, int playerId,
255         int wallsLeft1, int wallsLeft2) {
256         Board copy = board.copy();
257         applyMove(copy, move);
258         int size = (int) Math.sqrt(copy.getTiles().size());
259         int newWallsLeft1 = wallsLeft1;
260         int newWallsLeft2 = wallsLeft2;
261         if (move.getMoveType() == MoveType.PLACE_WALL) {
262             if (move.getPlayerId() == 1) {
263                 --newWallsLeft1;
264             } else {
265                 --newWallsLeft2;
266             }
267         }
268         int preference = playerId == rootPlayerId ?
269             movementPreference(move, board, playerId, size,
270                 recentPositions) : 0;
271         return evaluate(copy, playerId, size, newWallsLeft1,
272             newWallsLeft2) + preference;
273     }
274
275     private double normalizedEvaluate(Board board, int playerId
276         , int wallsLeft1, int wallsLeft2) {
277         int size = (int) Math.sqrt(board.getTiles().size());
278         return Math.tanh(evaluate(board, playerId, size,
279             wallsLeft1, wallsLeft2) / 500.0) * 0.15;
280     }
281
282     private double robustChildScore(Node node) {
283         if (node.visits == 0) {
284             return 0;
285         }
286         return node.visits + node.wins / node.visits;
287     }
288
289     private double bestWinRate(Node root) {
290         return root.children.stream().mapToDouble(n -> n.visits
291             == 0 ? 0 : n.wins / n.visits).max().orElse(0);
292     }

```

```

286
287     public boolean isWin(Board board, int playerId, int size) {
288         int row = getCurrentPosition(board, playerId).row();
289         return (playerId == 1 && row == 0) || (playerId == 2 &&
                row == size - 1);
290     }
291
292     static class Node {
293         Board board;
294         Node parent;
295         List<Node> children = new ArrayList<>();
296         List<Move> untriedMoves = new ArrayList<>();
297         Move move;
298
299         int visits = 0;
300         double wins = 0;
301
302         int player;
303         int walls1, walls2;
304
305         Node(Board board, Node parent, Move move, int player,
                int w1, int w2) {
306             this.board = board;
307             this.parent = parent;
308             this.move = move;
309             this.player = player;
310             this.walls1 = w1;
311             this.walls2 = w2;
312         }
313
314         int getWalls() {
315             return player == 1 ? walls1 : walls2;
316         }
317     }
318 }

```

Реализация обучения весов функции оценки

Листинг Е.1 — Класс EvaluationWeightsStore

```

1 public final class EvaluationWeightsStore {
2     private static final Path FILE = Path.of("data", "
        evaluation-weights.properties");
3     private static final Path HISTORY_FILE = Path.of("data", "
        evaluation-weights-history.csv");
4     private static EvaluationWeights current;
5
6     private EvaluationWeightsStore() {}
7
8     public static synchronized EvaluationWeights current() {
9         if (current == null) {
10             current = load();
11         }
12         return current;
13     }
14
15     public static synchronized EvaluationTrainingUpdate
        learnFromGame(List<EvaluationLearningSample> samples,
        int winnerId, String source) {
16         EvaluationWeights beforeWeights = current();
17         if (samples.isEmpty() || winnerId == 0) {
18             return new EvaluationTrainingUpdate(false, winnerId
                , samples.size(), new int[7], beforeWeights,
                beforeWeights);
19         }
20
21         int[] gradient = new int[7];
22         for (EvaluationLearningSample sample : samples) {
23             int reward = sample.playerId() == winnerId ? 1 :
                -1;
24             EvaluationFeatures delta = sample.delta();
25             gradient[0] += reward * delta.pathAdvantage();
26             gradient[1] += reward * delta.myMobility();
27             gradient[2] += reward * delta.enemyMobilityPenalty

```



```

        ();
28         gradient[3] += reward * delta.wallAdvantage();
29         gradient[4] += reward * delta.progressAdvantage();
30         gradient[5] += reward * delta.myEndgame();
31         gradient[6] += reward * delta.enemyEndgamePenalty()
            ;
32     }
33
34     if (isZero(gradient)) {
35         return new EvaluationTrainingUpdate(false, winnerId
            , samples.size(), gradient, beforeWeights,
            beforeWeights);
36     }
37
38     current = beforeWeights.adjustedByGameGradient(gradient
        , 1, samples.size());
39     save(current);
40     appendHistory(source, winnerId, samples.size(),
        gradient, beforeWeights, current);
41     return new EvaluationTrainingUpdate(true, winnerId,
        samples.size(), gradient, beforeWeights, current);
42 }
43
44 private static EvaluationWeights load() {
45     if (!Files.exists(FILE)) {
46         EvaluationWeights defaults = EvaluationWeights.
            defaults();
47         save(defaults);
48         return defaults;
49     }
50
51     Properties properties = new Properties();
52     try (Reader reader = Files.newBufferedReader(FILE,
        StandardCharsets.UTF_8)) {
53         properties.load(reader);
54         return new EvaluationWeights(
55             readInt(properties, "pathAdvantage",
                EvaluationWeights.defaults().
                    pathAdvantage()),
56             readInt(properties, "myMobility",

```

```

        EvaluationWeights.defaults().myMobility
        ()),
57         readInt(properties, "enemyMobility",
            EvaluationWeights.defaults().
            enemyMobility()),
58         readInt(properties, "wallAdvantage",
            EvaluationWeights.defaults().
            wallAdvantage()),
59         readInt(properties, "progressAdvantage",
            EvaluationWeights.defaults().
            progressAdvantage()),
60         readInt(properties, "myEndgame",
            EvaluationWeights.defaults().myEndgame()
            ),
61         readInt(properties, "enemyEndgame",
            EvaluationWeights.defaults().
            enemyEndgame()),
62         readLong(properties, "samples", 0)
63     );
64     } catch (IOException | IllegalArgumentException e) {
65         EvaluationWeights defaults = EvaluationWeights.
            defaults();
66         save(defaults);
67         return defaults;
68     }
69 }
70
71 private static void save(EvaluationWeights weights) {
72     try {
73         Files.createDirectories(FILE.getParent());
74         Files.writeString(FILE, serialize(weights),
            StandardCharsets.UTF_8);
75     } catch (IOException ignored) {
76         // Если файл недоступен, алгоритмы продолжают работать с
            весами в памяти.
77     }
78 }
79
80 private static String serialize(EvaluationWeights weights)
    {

```

```

81         return """
82         _____pathAdvantage=%d
83         _____myMobility=%d
84         _____enemyMobility=%d
85         _____wallAdvantage=%d
86         _____progressAdvantage=%d
87         _____myEndgame=%d
88         _____enemyEndgame=%d
89         _____samples=%d
90         _____""".formatted(
91             weights.pathAdvantage(),
92             weights.myMobility(),
93             weights.enemyMobility(),
94             weights.wallAdvantage(),
95             weights.progressAdvantage(),
96             weights.myEndgame(),
97             weights.enemyEndgame(),
98             weights.samples()
99         );
100     }
101
102     private static void appendHistory(String source, int
        winnerId, int sampleCount, int[] gradient,
        EvaluationWeights before, EvaluationWeights after) {
103         try {
104             Files.createDirectories(HISTORY_FILE.getParent());
105             if (!Files.exists(HISTORY_FILE)) {
106                 Files.writeString(HISTORY_FILE, historyHeader()
107                     , StandardCharsets.UTF_8);
108             }
109             Files.writeString(HISTORY_FILE, historyRow(source,
110                 winnerId, sampleCount, gradient, before, after),
111                 StandardCharsets.UTF_8, StandardOpenOption.
112                     APPEND);
113         } catch (IOException ignored) {
114             // История полезна для диплома, но недоступность файла не
115             должна ломать обучение.
116         }
117     }
118 }

```

```

115     private static String historyHeader() {
116         return "timestamp,source,winnerId,sampleCount,"
117             + "gradientPath,gradientMyMobility,
                gradientEnemyMobility,gradientWall,
                gradientProgress,gradientMyEndgame,
                gradientEnemyEndgame,"
118             + "beforePath,beforeMyMobility,
                beforeEnemyMobility,beforeWall,
                beforeProgress,beforeMyEndgame,
                beforeEnemyEndgame,beforeSamples,"
119             + "afterPath,afterMyMobility,afterEnemyMobility
                ,afterWall,afterProgress,afterMyEndgame,
                afterEnemyEndgame,afterSamples\n";
120     }
121
122     private static String historyRow(String source, int
        winnerId, int sampleCount, int[] gradient,
        EvaluationWeights before, EvaluationWeights after) {
123         return String.join(", ",
124             LocalDateTime.now().toString(),
125             escape(source),
126             Integer.toString(winnerId),
127             Integer.toString(sampleCount),
128             Integer.toString(gradient[0]),
129             Integer.toString(gradient[1]),
130             Integer.toString(gradient[2]),
131             Integer.toString(gradient[3]),
132             Integer.toString(gradient[4]),
133             Integer.toString(gradient[5]),
134             Integer.toString(gradient[6]),
135             Integer.toString(before.pathAdvantage()),
136             Integer.toString(before.myMobility()),
137             Integer.toString(before.enemyMobility()),
138             Integer.toString(before.wallAdvantage()),
139             Integer.toString(before.progressAdvantage()),
140             Integer.toString(before.myEndgame()),
141             Integer.toString(before.enemyEndgame()),
142             Long.toString(before.samples()),
143             Integer.toString(after.pathAdvantage()),
144             Integer.toString(after.myMobility()),

```

```

145         Integer.toString(after.enemyMobility()),
146         Integer.toString(after.wallAdvantage()),
147         Integer.toString(after.progressAdvantage()),
148         Integer.toString(after.myEndgame()),
149         Integer.toString(after.enemyEndgame()),
150         Long.toString(after.samples())
151     ) + "\n";
152 }
153
154 private static String escape(String value) {
155     String safe = value == null ? "" : value.replace("\\"",
156         "\\\"");
157     return "\"" + safe + "\"";
158 }
159 private static int readInt(Properties properties, String
160     key, int fallback) {
161     return Integer.parseInt(properties.getProperty(key,
162         Integer.toString(fallback)));
163 }
164 private static long readLong(Properties properties, String
165     key, long fallback) {
166     return Long.parseLong(properties.getProperty(key, Long.
167         toString(fallback)));
168 }
169 private static boolean isZero(int[] values) {
170     for (int value : values) {
171         if (value != 0) {
172             return false;
173         }
174     }
175     return true;
176 }

```